

Measurement Systems

Lab Report 6

Instructor: Dr. Ali Sadighi

Author:
Roozbeh Amini - 810601291

February 19, 2026

Abstract

This report describes the design, calibration and verification of a temperature measurement system for 0–100 °C, built around a 10 kΩ NTC thermistor, a unity-gain difference amplifier and an Arduino Uno. The only reference thermometer available was a medical thermometer that reads between 32 and 43 °C, so all intermediate calibration points had to be taken in that range. The 0 °C point came from an ice bath. The high-end point came from boiling water, whose temperature was estimated as about 90 °C because of Tehran's altitude and the cooling of the water after boiling. Eight calibration points were recorded at 50 Hz for 60 s each, and the mean of the last 10 s of each record was used as the steady-state value.

The ice bath reading, $V_{\mathrm{th}}(0\,^{\circ}\mathrm{C}) = 3.73\,\mathrm{V}$, was used as the reference voltage of the difference amplifier (Figure 8). Fitting the β-parameter model gave $\beta = 3578.29\,\mathrm{K}$ and $R_0 = 9835.46\,\Omega$. Evaluating the model at 100 °C gives $V_{\mathrm{th}}(100\,^{\circ}\mathrm{C}) = 0.405\,\mathrm{V}$ and an amplifier output of 3.325 V. This value was used as the external ADC reference ($V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$), so the full ADC range covers 0–100 °C with a resolution of 3.25 mV per step, 33.5% finer than with the default 5 V reference. The 1 MΩ amplifier input resistors are about 134 times the worst-case source impedance of the divider (7 468 Ω), well above the usual 10× rule. In the final test, the system read +0.44 °C in the ice bath and 34.32 °C in a bath where the medical thermometer read 34.4 °C, a difference of 0.08 °C.

Contents

1 Introduction **5**
1.1 Session Objectives 5

2 Theoretical Background **5**
2.1 NTC Thermistor Characteristics 5
2.2 The β-Parameter Equation 6
2.2.1 Linearisation for Least-Squares Fitting 6
2.3 Voltage Divider 7
2.3.1 General Principle 7
2.3.2 Thermistor Voltage Divider 7
2.3.3 Potentiometer as an Adjustable Voltage Divider 7
2.3.4 Impedance Loading and the 10× Rule 8
2.4 Wheatstone Bridge 8
2.5 Difference Amplifier 9
2.6 Instrumentation Amplifier 9
2.7 ADC Reference Voltage and Resolution 10

3 Thermistor Calibration **10**
3.1 Calibration Strategy and Instrument Constraints 10
3.2 Reference Temperature Sources 11
3.3 Data Collection: 50 Hz Arduino Logger 12
3.4 Steady-State Analysis in MATLAB 13
3.5 Calibration Dataset 13
3.6 Steady-State Acquisition Plots and Observations 13
3.7 β-Model Fitting 19
3.8 Fitted Model Parameters and Calibration Curve 19
3.9 Calculation of the Design Voltages 20
3.9.1 Step 1: Ice Bath $\Rightarrow V_{\text{ref}} = 3.73\,\text{V}$ 20
3.9.2 Step 2: Extrapolation to 100 °C $\Rightarrow V_{\text{AREF}} = 3.325\,\text{V}$ 20
3.9.3 Voltage Table at All Calibration Points 21

4 Circuit Design and Signal Conditioning **21**
4.1 System Overview 21
4.2 Stage 1: Thermistor Voltage Divider 22
4.2.1 Circuit Description 22
4.2.2 Wiring on the Breadboard 22
4.3 Stage 2: Reference Voltage Generator (3.73 V) 23
4.3.1 Circuit Description 23
4.3.2 Wiring on the Breadboard 23
4.3.3 Setting the Potentiometer with the Live Voltage Reader 23
4.4 Stage 3: Unity-Gain Difference Amplifier 24
4.4.1 Circuit Description and Pin Assignment 24
4.4.2 Wiring on the Breadboard 24
4.4.3 Choice of 1 MΩ Resistors 25
4.4.4 Verification of the 10× Impedance Rule 25
4.4.5 Loading Error at 0 °C 25

4.4.6 Gain Stage vs. External ADC Reference . . . 26
4.5 Stage 4: External ADC Reference (3.325 V) . . . 26
4.5.1 Circuit Description . . . 26
4.5.2 Wiring on the Breadboard . . . 26
4.6 Full Assembled Circuit . . . 27

5 Firmware Implementation 29
5.1 AREF Safety Requirement . . . 29
5.2 Signal-Chain Inversion Logic . . . 29
5.3 Firmware Constants and Their Origins . . . 30

6 Results and Verification 30
6.1 Verification Procedure . . . 30
6.2 Verification Results . . . 30
6.3 Summary of Verification Accuracy . . . 31

7 Discussion 31
7.1 Model Validity and Calibration Density . . . 31
7.2 ADC Resolution Improvement . . . 32
7.3 Error Sources and Analysis . . . 33

8 Conclusion 34

Acknowledgements 35

Appendices 37

A Arduino Calibration Logger (50 Hz, 60 s) 37

B Live Voltage Reader 38

C MATLAB Calibration Acquisition Script 39

D MATLAB Model Fitting and Characteristic Plots 41

E Final Temperature Measurement Firmware 44

List of Figures

1 Typical NTC thermistor resistance vs. temperature characteristic [1]. . . . 5
2 The 10 kΩ NTC thermistor used in this experiment. 6
3 Voltage divider schematic: two series resistors between V_{cc} and GND produce a node voltage proportional to the lower resistance. 7
4 Wheatstone bridge schematic [1]. At balance the differential voltage is $V_D = 0$. When the bridge is unbalanced, as in sensor applications, V_D changes with the deviation of R_x from its nominal value. 8
5 Difference amplifier schematic [1]. With all four resistors equal the output is $V_{\text{out}} = V_2 - V_1$. 9
6 Three-op-amp instrumentation amplifier topology [1]. The gain is set by the single resistor R_G. 9
7 Calibration setup. Both sensors were kept in the bath until the medical thermometer showed a stable reading, and then the 60 s recording was started. 12
8 60 s recording at 0 °C (ice bath). The rising transient is the thermistor cooling from room temperature. Steady-state mean: $V_{\text{th}} = 3.7300$ V, $R_{\text{th}} = 29{,}498.05\,\Omega$. 14
9 60 s recording at 33.2 °C. Steady state: $R_{\text{th}} = 7{,}241.27\,\Omega$. 15
10 60 s recording at 34.4 °C. Steady state: $R_{\text{th}} = 6{,}832.37\,\Omega$. 15
11 60 s recording at 35.3 °C. Steady state: $R_{\text{th}} = 6{,}585.11\,\Omega$. 16
12 60 s recording at 37.0 °C. Steady state: $R_{\text{th}} = 6{,}135.16\,\Omega$. 16
13 60 s recording at 37.8 °C. Steady state: $R_{\text{th}} = 5{,}944.50\,\Omega$. 17
14 60 s recording at 38.2 °C. Steady state: $R_{\text{th}} = 5{,}892.11\,\Omega$. 17
15 60 s recording in boiling water (estimated 90 °C). The sharp initial drop is followed by a flat plateau. Steady state (last 10 s): $V_{\text{th}} = 0.5100$ V, $R_{\text{th}} = 1{,}135.84\,\Omega$. 18
16 Calibration curve: measured points (red) and fitted β-model (blue). The model is extrapolated to 100 °C to find V_{AREF}. 19
17 Expected thermistor characteristics from the fitted model over 0–100 °C. *Left:* resistance $R(T)$. *Right:* divider output $V_{\text{th}}(T)$ for $V_{cc} = 5$ V and $R_1 = 10$ kΩ. The calibration points are shown on both plots. 20
18 Stage 1 thermistor voltage divider wiring. The V_{th} node is connected by a jumper to the inverting input of Stage 3. 22
19 Stage 3 difference amplifier wiring. All four resistors are 1 MΩ. Inverting input: V_{th} from Stage 1. Non-inverting input: $V_{\text{ref}} = 3.73$ V from Stage 2. Output to Arduino A0. 24
20 Stage 4 AREF wiring, with the 5 kΩ protection resistor between the potentiometer wiper and the AREF pin. 27
21 Complete measurement circuit: (a) schematic and (b) breadboard. The breadboard follows the left-to-right order of the schematic. A walkthrough video is available at [4]. 28

1. Introduction

NTC (negative temperature coefficient) thermistors are common in mechatronic instrumentation because they are sensitive, cheap and small [1]. Their resistance changes with temperature in a strongly nonlinear way, so a calibration model and some signal conditioning are needed before the signal can be read by a microcontroller ADC.

This report follows the workflow given in the course manual [1]: characterising the thermistor, designing the signal conditioning circuit and checking its loading, fitting a model in MATLAB, and running calibrated firmware on an Arduino Uno. Calibration is presented before the circuit design, because the two design voltages come from it: the difference amplifier reference $V_{\text{ref}} = 3.73\,\text{V}$ and the external ADC reference $V_{\text{AREF}} = 3.325\,\text{V}$ are both based on the calibration measurements and the fitted model. Neither can be taken from the thermistor datasheet alone.

1.1. Session Objectives

1. Understand how NTC thermistors work and how the β-parameter model describes them.
2. Design and build a voltage divider, a difference amplifier and an external ADC reference for the thermistor signal.
3. Collect calibration points, fit the β model by linearised least squares in MATLAB, and use the model to choose the ADC reference voltage.
4. Upload calibrated firmware to the Arduino Uno and check that the system works as a thermometer.

2. Theoretical Background

2.1. NTC Thermistor Characteristics

An NTC thermistor is a semiconductor resistor whose resistance drops sharply as temperature rises [1]. At higher temperatures more electrons are thermally excited into the conduction band, which increases the conductivity and lowers the resistance. This makes the thermistor very sensitive, but also strongly nonlinear, as Figure 1 shows. The thermistor used here (Figure 2) has a nominal resistance of $10\,\text{k}\Omega$ at $25\,^{\circ}\text{C}$.

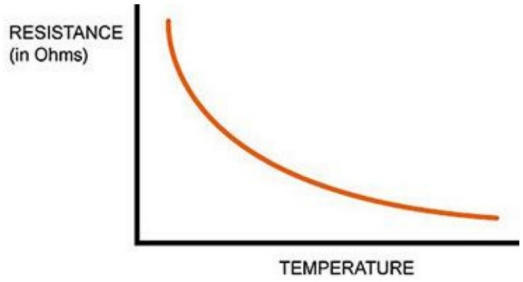


Figure 1: Typical NTC thermistor resistance vs. temperature characteristic [1].



Figure 2: The 10 kΩ NTC thermistor used in this experiment.

2.2. The β-Parameter Equation

The lab manual [1] uses the β-parameter equation to describe the thermistor:

$$\frac{1}{T} = \frac{1}{T_0} + \frac{1}{\beta} \ln \frac{R}{R_0} \tag{1}$$

Solving for resistance:

$$\boxed{R(T) = R_0 \exp\left[\beta\left(\frac{1}{T} - \frac{1}{T_0}\right)\right]} \tag{2}$$

where $T_0 = 298.15\,\mathrm{K}$ (25 °C) is the reference temperature, R_0 is the resistance at T_0, and β is a material constant, usually 3000–4000 K for NTC thermistors. Solving for temperature instead gives:

$$T = \frac{1}{\dfrac{1}{T_0} + \dfrac{1}{\beta} \ln \dfrac{R}{R_0}} \tag{3}$$

2.2.1. Linearisation for Least-Squares Fitting

Taking the logarithm of Equation (2) gives a linear relation:

$$\underbrace{\ln R}_{y} = \underbrace{\ln R_0}_{\text{intercept}} + \beta \underbrace{\left(\frac{1}{T} - \frac{1}{T_0}\right)}_{x} \tag{4}$$

With $x_i = 1/T_i - 1/T_0$ and $y_i = \ln R_i$, this is the straight line $y = \beta\, x + \ln R_0$. MATLAB's `polyfit(x,y,1)` fits it by ordinary least squares and returns the slope $\hat{\beta}$ and the intercept $\ln \hat{R}_0$, so $\hat{R}_0 = \exp(\text{intercept})$.

2.3. Voltage Divider

2.3.1. General Principle

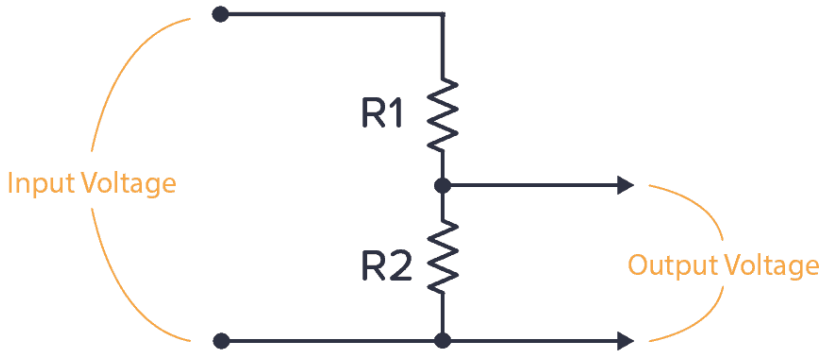


Figure 3: Voltage divider schematic: two series resistors between V_{cc} and GND produce a node voltage proportional to the lower resistance.

As shown in Figure 3, two resistors in series divide the supply voltage:

$$V_{\text{out}} = V_{cc} \cdot \frac{R_2}{R_1 + R_2} \tag{5}$$

Any load connected to the output node is in parallel with R_2. This lowers the effective lower resistance and pulls V_{out} below its ideal value. This *loading* effect is calculated in Section 2.3.4.

2.3.2. Thermistor Voltage Divider

With the thermistor as the lower resistor ($R_2 = R_{\text{th}}$, $R_1 = 10\,\text{k}\Omega$, $V_{cc} = 5\,\text{V}$):

$$V_{\text{th}} = V_{cc} \cdot \frac{R_{\text{th}}}{R_1 + R_{\text{th}}}, \qquad R_{\text{th}} = R_1 \cdot \frac{V_{\text{th}}}{V_{cc} - V_{\text{th}}} \tag{6}$$

Since the NTC resistance falls with temperature, V_{th} falls too. At $0\,^\circ\text{C}$ ($R_{\text{th}} \approx 29{,}500\,\Omega$) the node is at about $3.73\,\text{V}$, and at $100\,^\circ\text{C}$ ($R_{\text{th}} \approx 881\,\Omega$, from the model) it drops to $0.405\,\text{V}$.

2.3.3. Potentiometer as an Adjustable Voltage Divider

A potentiometer splits its total resistance R_{total} into an upper part R_{top} and a lower part R_{bot}. With its two ends connected to V_{cc} and GND, the wiper voltage is:

$$V_{\text{wiper}} = V_{cc} \cdot \frac{R_{\text{bot}}}{R_{\text{total}}} \tag{7}$$

Turning the adjustment screw changes R_{bot}, so the wiper voltage can be set anywhere between 0 and V_{cc}. Two potentiometers were used this way: potentiometer 1 for the

3.73 V amplifier reference and potentiometer 2 for the 3.325 V ADC reference. Both were set using a live voltage reader sketch on the Arduino.

2.3.4. Impedance Loading and the 10× Rule

The Thévenin output impedance of the divider is:

$$R_{\mathrm{Th}} = R_1 \parallel R_{\mathrm{th}} = \frac{R_1 R_{\mathrm{th}}}{R_1 + R_{\mathrm{th}}} \tag{8}$$

When a load $R_L \gg R_{\mathrm{Th}}$ is connected, the voltage error is approximately:

$$\varepsilon_V \approx V_{\mathrm{th}} \cdot \frac{R_{\mathrm{Th}}}{R_L} \tag{9}$$

The usual 10× rule limits this error to about 10% of V_{th} (about 9% if calculated exactly):

$$\boxed{R_L \geq 10\, R_{\mathrm{Th}}} \tag{10}$$

2.4. Wheatstone Bridge

A Wheatstone bridge (Figure 4) measures an unknown resistance by balancing two legs of a four-resistor network [1]. At balance:

$$R_x = \frac{R_2}{R_1} \cdot R_3 \tag{11}$$

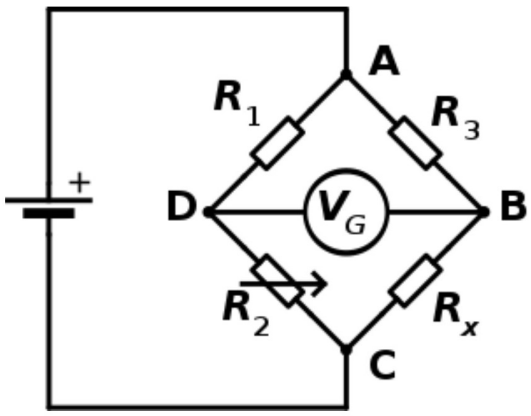


Figure 4: Wheatstone bridge schematic [1]. At balance the differential voltage is $V_D = 0$. When the bridge is unbalanced, as in sensor applications, V_D changes with the deviation of R_x from its nominal value.

When the bridge is not balanced, the voltage between its two midpoints depends on how far R_x is from its reference value. The circuit in this experiment gets the same kind of differential measurement in a simpler way: a difference amplifier subtracts V_{th} from a fixed reference voltage, so a full four-resistor bridge is not needed.

2.5. Difference Amplifier

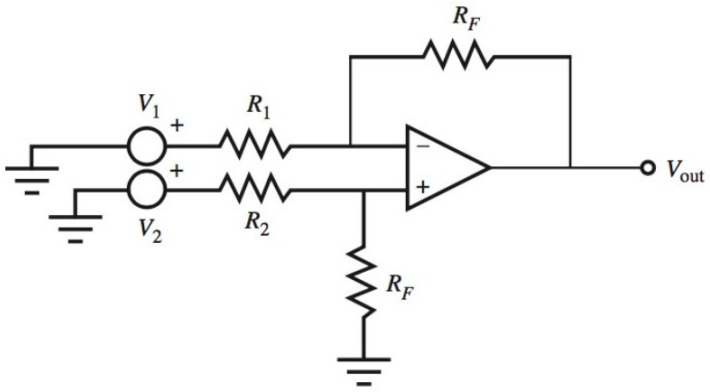


Figure 5: Difference amplifier schematic [1]. With all four resistors equal the output is $V_{\text{out}} = V_2 - V_1$.

The difference amplifier (Figure 5) subtracts two voltages, with a gain set by the resistor ratio [1, 2]:

$$V_{\text{out}} = \frac{R_F}{R_{\text{in}}}(V_2 - V_1) \tag{12}$$

With all four resistors equal ($R_F = R_{\text{in}}$), the gain is one:

$$\boxed{V_{\text{out}} = V_2 - V_1} \tag{13}$$

Here $V_2 = V_{\text{ref}} = 3.73\,\text{V}$ (potentiometer 1) and $V_1 = V_{\text{th}}$ (the thermistor divider node), so the output shows how far the thermistor voltage has moved from its 0 °C value.

2.6. Instrumentation Amplifier

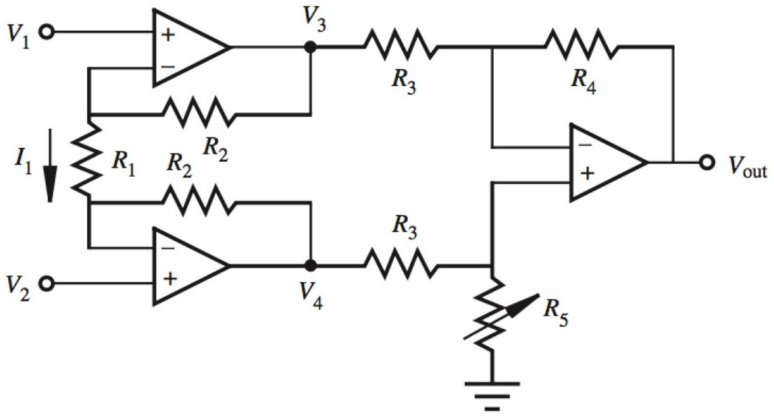


Figure 6: Three-op-amp instrumentation amplifier topology [1]. The gain is set by the single resistor R_G.

An instrumentation amplifier (Figure 6) has buffered inputs, so its input impedance is very high and loading is not a concern. Its gain is set by a single resistor:

$$G = 1 + \frac{2R}{R_G} \tag{14}$$

Alternative Implementations

Instrumentation amplifier (e.g. AD620 [3]): would remove the loading error and allow a gain above 1 with a single external resistor.
Dual DC supply: with a $\pm 9\,\mathrm{V}$ or $\pm 12\,\mathrm{V}$ supply, the op-amp output could swing over the whole ADC range, and the external AREF potentiometer would not be needed. Section 4.4.6 explains why a gain stage was not used instead.

2.7. ADC Reference Voltage and Resolution

The Arduino Uno's 10-bit ADC maps [0, V_{AREF}] to [0, 1023] [1]:

$$V_{\mathrm{measured}} = \frac{\mathrm{ADC}}{1023} \cdot V_{\mathrm{AREF}}, \qquad \Delta V = \frac{V_{\mathrm{AREF}}}{1023} \tag{15}$$

With the default 5 V reference and a signal between 0 and 3.325 V, only 680 of the 1023 steps (66.5%) are used. With $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ all 1023 steps are used, and the step size drops to 3.25 mV, a 33.5% improvement over the default.

3. Thermistor Calibration

Calibration comes before the circuit design in this report because both design voltages come from it. $V_{\mathrm{ref}} = 3.73\,\mathrm{V}$ is the ice bath reading, and $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ is the fitted model's prediction at 100 °C. Neither value can be read from the thermistor datasheet.

3.1. Calibration Strategy and Instrument Constraints

The calibration was limited by the reference instrument available: a standard digital medical thermometer. It is easy to get and accurate around body temperature, but it only reads between about 32 °C and 43 °C. Outside this range it shows `Lo` (below 32 °C) or `Hi` (above 43 °C) and cannot be used as a reference.

This is why the calibration points are spread the way they are:

- **Intermediate points (33–38 °C):** taken in mixtures of cold and hot water, inside the thermometer's range. The six points are close together only because of this limit.
- **0 °C (ice bath):** taken without the thermometer. The melting point of ice at atmospheric pressure is a fixed point and needs no reference instrument.
- **≈90 °C (boiling water):** taken without the thermometer, since this temperature is above its range. The temperature was therefore estimated, as explained in Section 3.2.

Ideally the points would be spread evenly over 0–100 °C (for example at 0, 15, 30, 45, 60, 75 and 90 °C), but that would need a calibrated thermocouple or platinum RTD. Section 7.1 discusses how this affects the model.

3.2. Reference Temperature Sources

1. **Ice bath, 0 °C (fixed point).** A well-stirred mixture of crushed ice and distilled water. Its temperature is within about 0.05 °C of 0 °C, so no reference instrument is needed. This was the first measurement, and its steady-state reading ($V_{\text{th}} = 3.73\,\text{V}$, Figure 8) sets V_{ref} for the difference amplifier.
2. **Boiling water, estimated 90 °C.** Water was boiled in an electric kettle, the probe was put in right away, and the 60 s recording was started. The water temperature could not be measured, because it is above the thermometer's range, so it was estimated as 90 °C instead of 100 °C, for two reasons:
   - **Tehran's altitude.** At about 1200–1500 m above sea level, the atmospheric pressure is about 0.83–0.87 atm. The boiling point of water drops by about 0.34 °C per 100 m, so it is 4–5 °C lower here.
   - **Cooling after the kettle is unplugged.** The water starts losing heat as soon as the kettle is unplugged, so it was assumed to be a few degrees below its boiling point during the measurement.

   Because this value is an estimate, its uncertainty is listed as an error source in Section 7.3. The point is used as the high-end anchor of the β fit. The ADC reference is not taken from this point directly; it comes from extrapolating the fitted model to 100 °C (Section 3.9).
3. **Mixed baths, 33–38 °C.** Cold and hot water mixed in different proportions, with the medical thermometer in the same bath. Only readings inside the thermometer's 32–43 °C range were used.



(a) Medical thermometer (valid range 32–43 °C).

(b) Thermistor and thermometer in the same bath.

Figure 7: Calibration setup. Both sensors were kept in the bath until the medical thermometer showed a stable reading, and then the 60 s recording was started.

3.3. Data Collection: 50 Hz Arduino Logger

For each calibration point, an Arduino sketch sampled the thermistor voltage at 50 Hz (every 20 ms) for 60 s and sent the data over serial at 115200 baud as comma-separated values. The sampling is timed with `millis()` instead of `delay()`, so the sample spacing does not drift:

Listing 1: Core 50 Hz acquisition loop. The resistance is computed on the Arduino by inverting the voltage divider (Eq. (6)).

```
while (millis() - t0 < DURATION_MS) {
  if (millis() >= next) {
    next += Ts_ms;                              // 20 ms -> 50 Hz
    float t_s = (millis() - t0) / 1000.0f;
    int   adc = analogRead(THERM_PIN);
    float v   = adc * VREF / 1023.0f;
    float Rth = (v > 0.01f && v < VREF - 0.01f)
                  ? R_FIXED * v / (VREF - v)
                  : 0.0f;
    Serial.print(t_s, 4); Serial.print(",");
    Serial.print(adc);    Serial.print(",");
    Serial.print(v, 6);   Serial.print(",");
    Serial.println(Rth, 2);
  }
}
```

The full logger sketch is given in Appendix A.

3.4. Steady-State Analysis in MATLAB

MATLAB reads each 3000-sample record through the `serialport` interface. The last 10 s (500 samples) are used as the steady-state window, since by then the thermistor has settled at the bath temperature. Their mean and standard deviation are computed as follows:

Listing 2: Steady-state extraction in MATLAB: the last `steady_win` seconds are averaged.

```
t_max  = max(T);
ss     = T >= (t_max - steady_win);   % last 10 s
v_mean = mean(V(ss));
r_mean = mean(R(ss));
r_std  = std(R(ss));
```

Each recording is plotted with the raw 50 Hz signal (grey), a 1 s moving average (coloured), the steady-state window (green) and the mean (dashed line), so any transient that has not died out can be seen before a point is accepted. The full script is given in Appendix C.

3.5. Calibration Dataset

Table 1: The eight calibration points. R_{th} is the mean of the last 10 s of each 60 s record.

Pt	T_{ref} (°C)	T (K)	R_{th} (Ω)	**Source**
1	0.0	273.15	29 498.05	Ice bath
2	33.2	306.35	7 241.27	Mixed + therm.
3	34.4	307.55	6 832.37	Mixed + therm.
4	35.3	308.45	6 585.11	Mixed + therm.
5	37.0	310.15	6 135.16	Mixed + therm.
6	37.8	310.95	5 944.50	Mixed + therm.
7	38.2	311.35	5 892.11	Mixed + therm.
8	90.0	363.15	1 135.84	Boiling water (est.)

3.6. Steady-State Acquisition Plots and Observations

Figures 8–15 show the 60 s recordings for all eight calibration points. Each figure shows the raw 50 Hz signal (grey), a 1 s moving average (coloured line), the steady-state window (green shading) and the steady-state mean (dashed line).

Ice bath, 0 °C (Figure 8). The thermistor went into the ice bath from room temperature (about 25 °C). Since the NTC resistance increases as the temperature falls, V_{th} rises during the first 20–25 s as the thermistor cools, from about 2.48 V at 25 °C to 3.73 V at 0 °C, a change of +1.25 V. The last 10 s show a stable, low-noise plateau with a mean of $V_{\text{th}} = 3.7300$ V, which is used directly as $V_{\text{ref}} = 3.73$ V in the difference amplifier (Step 1, Section 3.9).

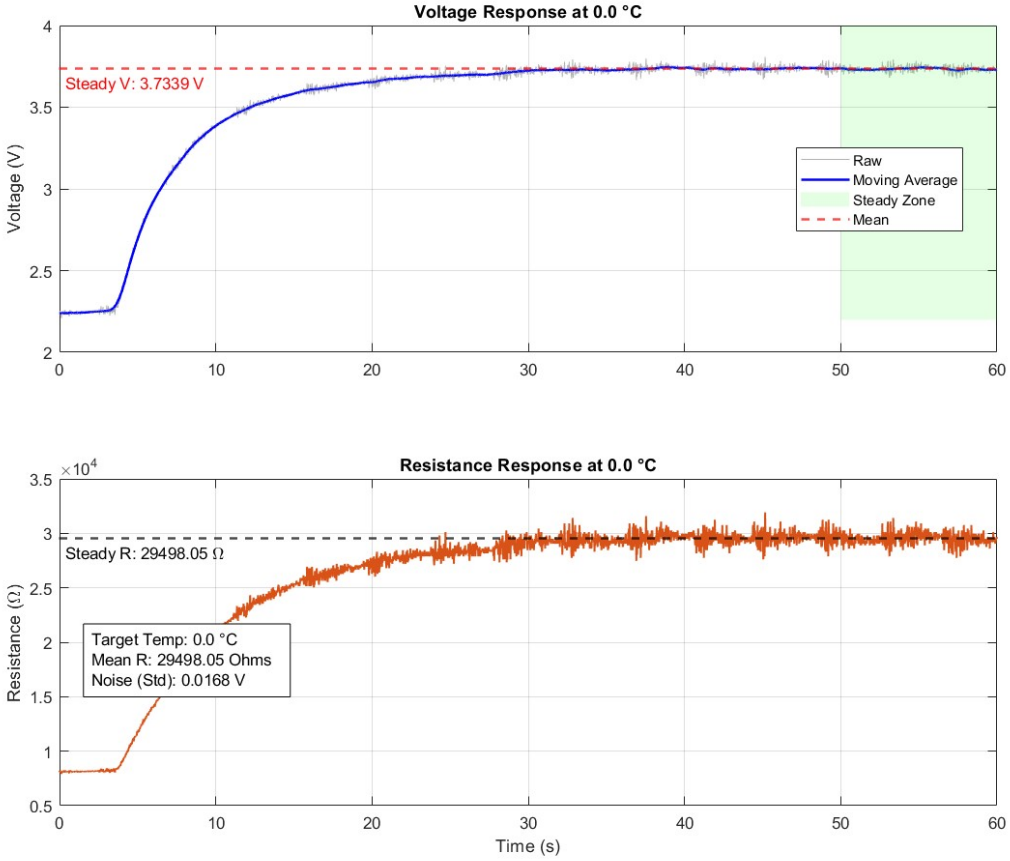


Figure 8: 60 s recording at 0 °C (ice bath). The rising transient is the thermistor cooling from room temperature. Steady-state mean: $V_{\text{th}} = 3.7300\,\text{V}$, $R_{\text{th}} = 29{,}498.05\,\Omega$.

Mixed baths, 33–38 °C (Figures 9–14). These recordings settle much faster than the ice bath, because the thermistor starts only 8–13 °C away from the bath temperature, compared with 25 °C for the ice bath. The reading settles within 5–10 s, and the remaining 50 s are flat. As the bath temperature increases, V_{th} decreases from about 2.10 V at 33.2 °C to about 1.85 V at 38.2 °C, as expected for an NTC thermistor. The small standard deviation in each 10 s window shows that the thermistor had settled and the ADC signal was stable.

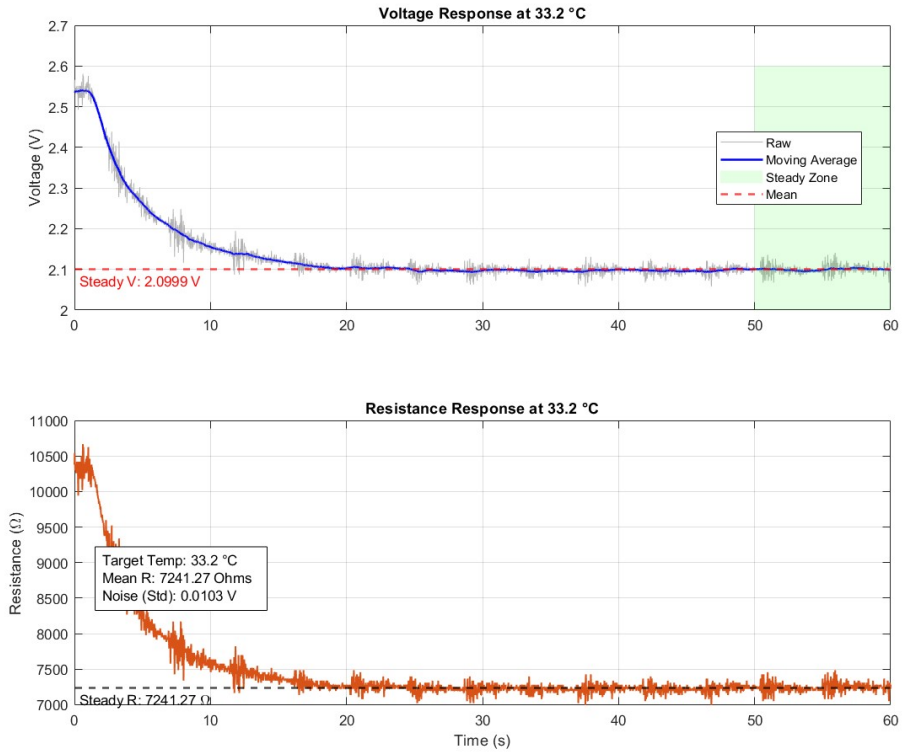


Figure 9: 60 s recording at 33.2 °C. Steady state: $R_{\text{th}} = 7{,}241.27\,\Omega$.

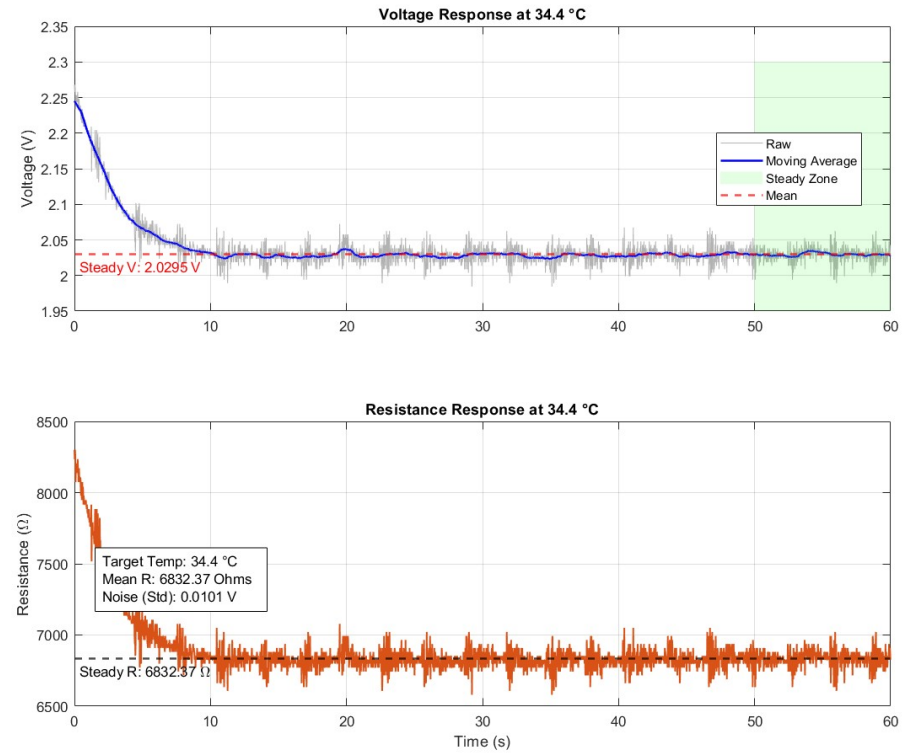


Figure 10: 60 s recording at 34.4 °C. Steady state: $R_{\text{th}} = 6{,}832.37\,\Omega$.

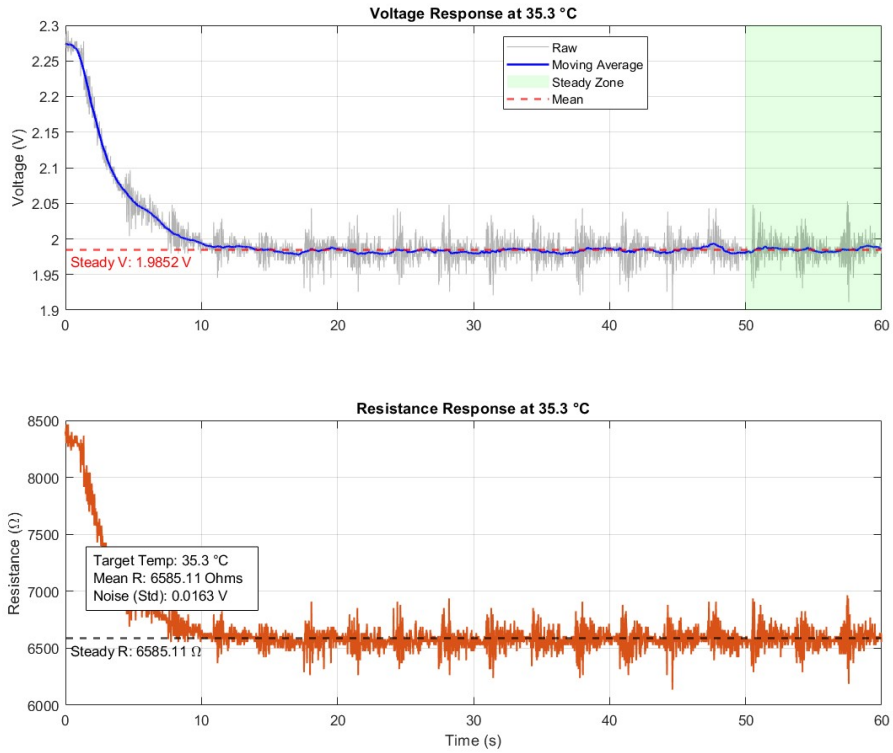


Figure 11: 60 s recording at 35.3 °C. Steady state: $R_{\text{th}} = 6{,}585.11\,\Omega$.

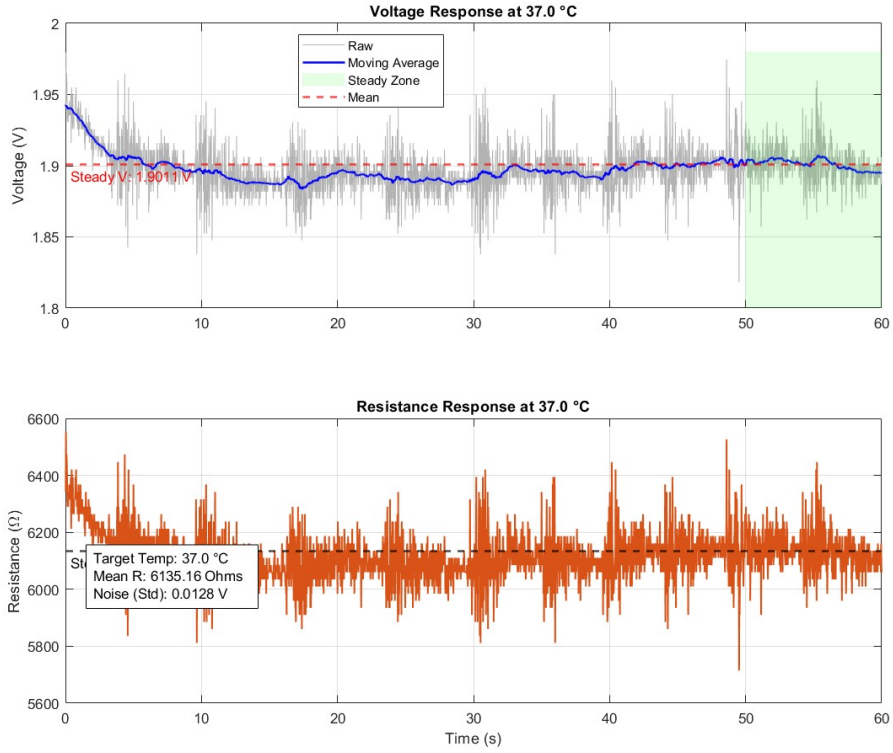


Figure 12: 60 s recording at 37.0 °C. Steady state: $R_{\text{th}} = 6{,}135.16\,\Omega$.

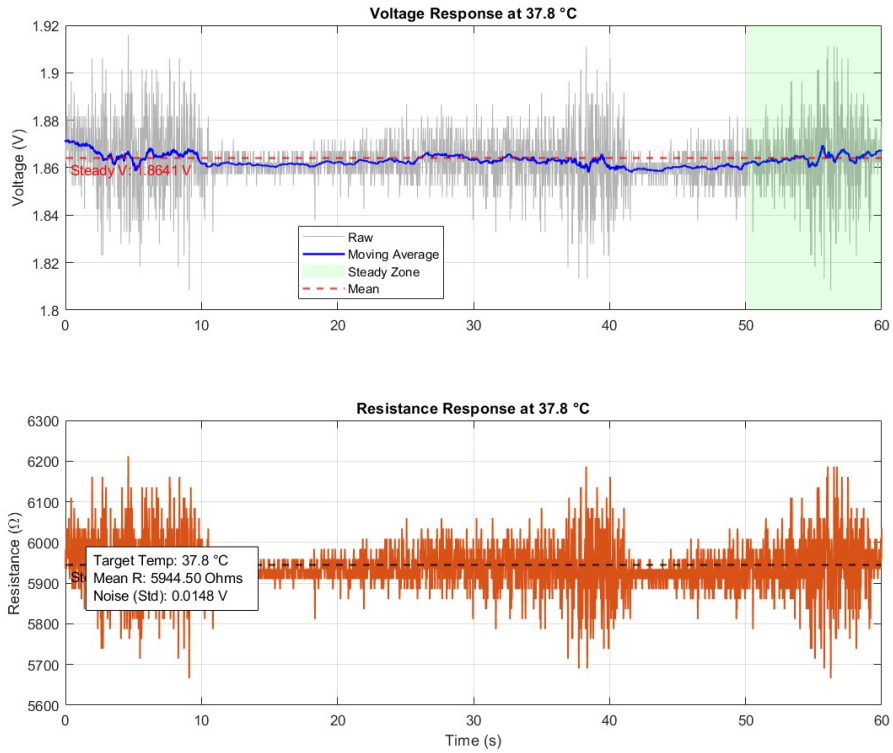


Figure 13: 60 s recording at 37.8 °C. Steady state: $R_{\text{th}} = 5{,}944.50\,\Omega$.

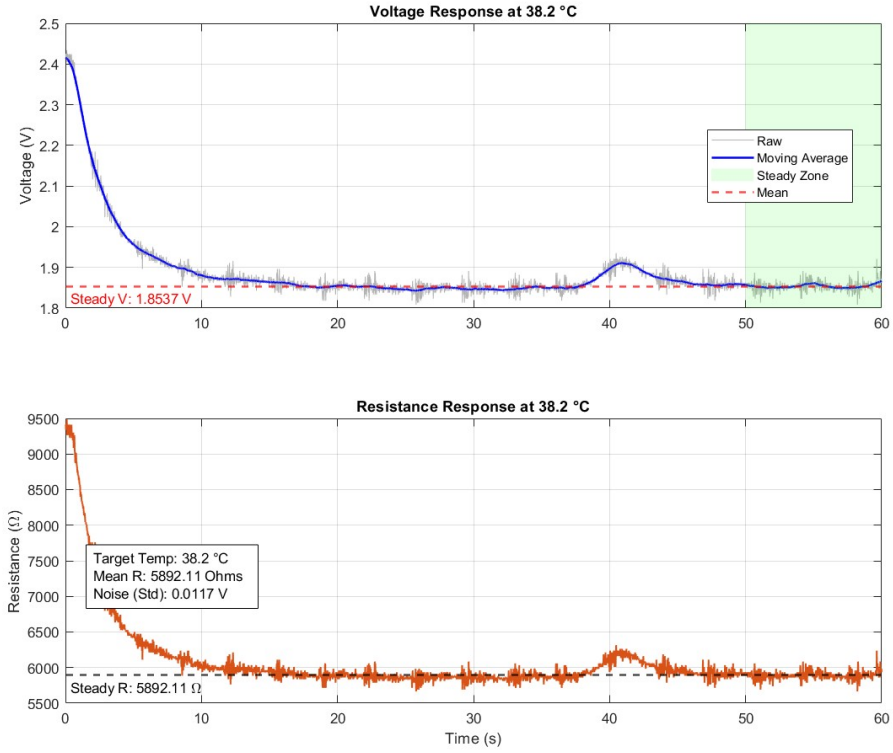


Figure 14: 60 s recording at 38.2 °C. Steady state: $R_{\text{th}} = 5{,}892.11\,\Omega$.

Boiling water, about 90 °C (Figure 15). Here the thermistor goes from room temperature into very hot water. The resistance drops sharply in the first 5 s, and V_{th} falls from about 2.48 V to about 0.51 V, a drop of about 1.97 V. After about 15 s the reading levels off, and the last 10 s show a stable plateau, which means the thermistor has reached the bath temperature. The steady-state mean is $V_{\text{th}} = 0.5100\,\text{V}$ and $R_{\text{th}} = 1{,}135.84\,\Omega$. This point is the high-end anchor of the β fit. The AREF voltage is found later by extrapolating the fitted model to 100 °C (Section 3.9).

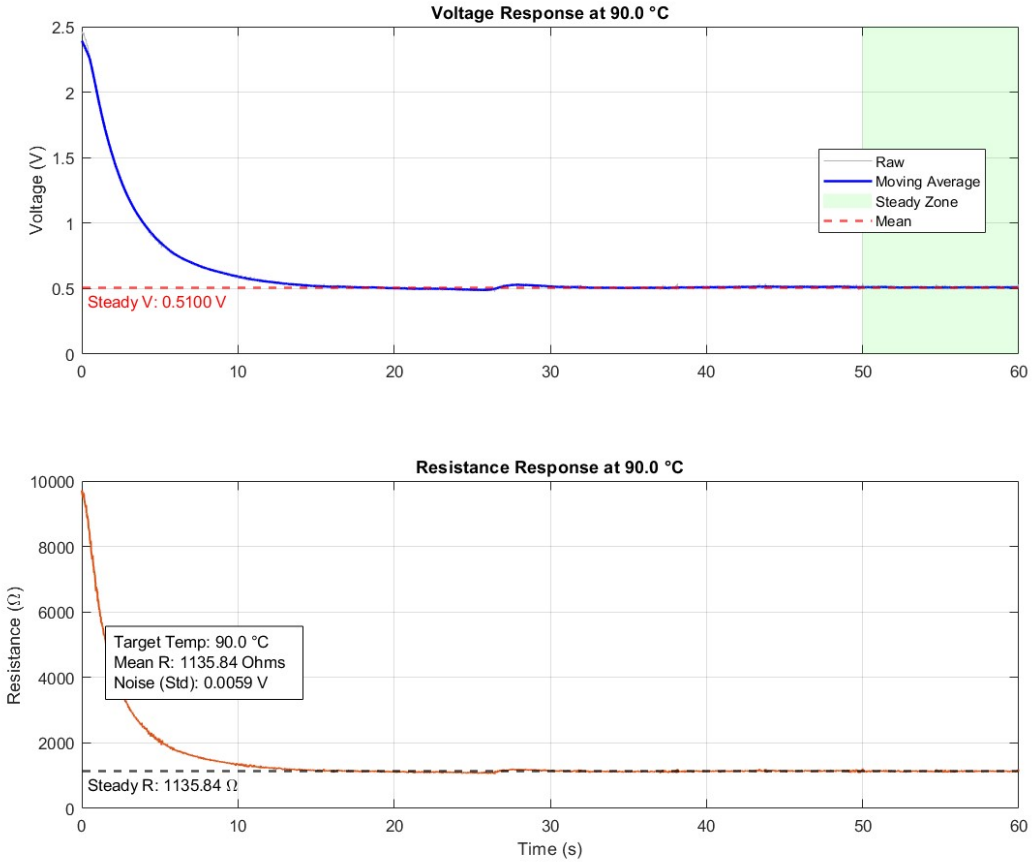


Figure 15: 60 s recording in boiling water (estimated 90 °C). The sharp initial drop is followed by a flat plateau. Steady state (last 10 s): $V_{\text{th}} = 0.5100\,\text{V}$, $R_{\text{th}} = 1{,}135.84\,\Omega$.

3.7. β-Model Fitting

The calibration pairs $(T_i,\ R_i)$ from Table 1 are converted to the linear form of Equation (4) and fitted in MATLAB:

Listing 3: Linearised β-model fit.

```
X     = (1 ./ T_K) - (1 / T0);
Y     = log(R_meas);
p     = polyfit(X, Y, 1);
Beta  = p(1);
R0    = exp(p(2));
```

The full script is given in Appendix D.

3.8. Fitted Model Parameters and Calibration Curve

$$\boxed{\beta = 3578.29\,\mathrm{K}, \qquad R_0 = 9835.46\,\Omega} \tag{16}$$

The fitted $\beta = 3578\,\mathrm{K}$ is inside the usual 3000–4000 K range for NTC thermistors, and $R_0 = 9835.46\,\Omega$ is within 1.65% of the nominal $10\,\mathrm{k}\Omega$, so both values are reasonable. Figure 16 shows the fitted curve with all eight calibration points.

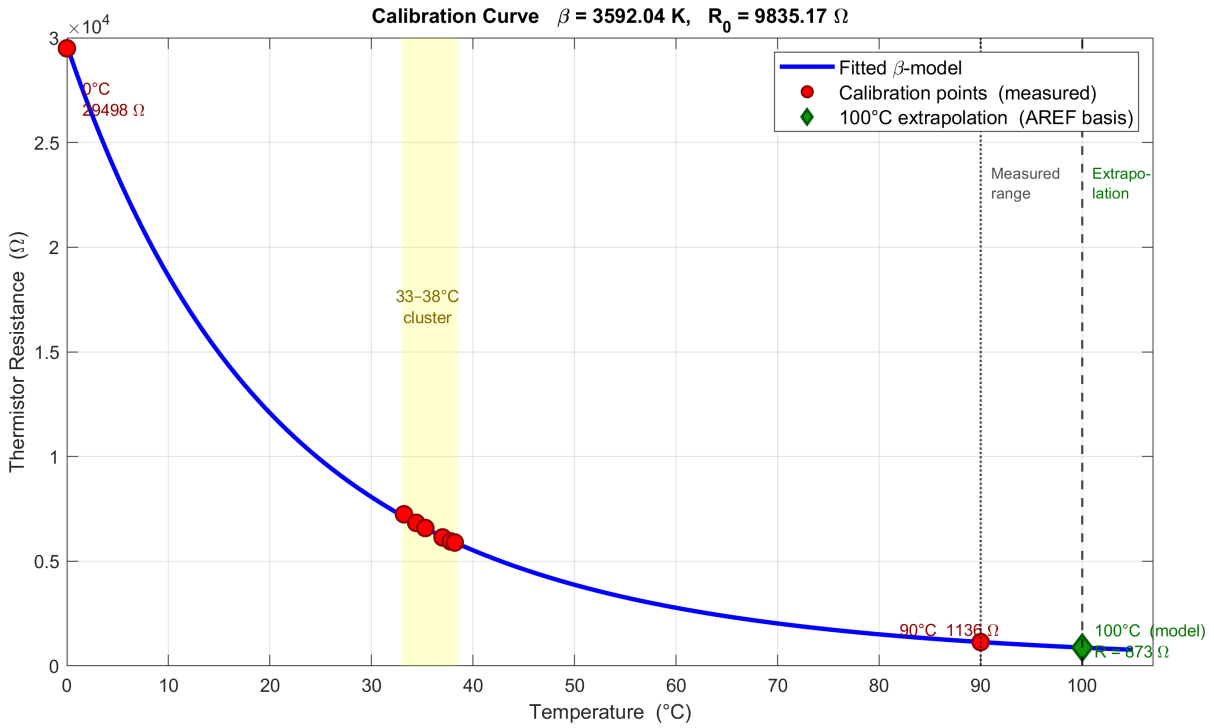


Figure 16: Calibration curve: measured points (red) and fitted β-model (blue). The model is extrapolated to 100 °C to find V_{AREF}.

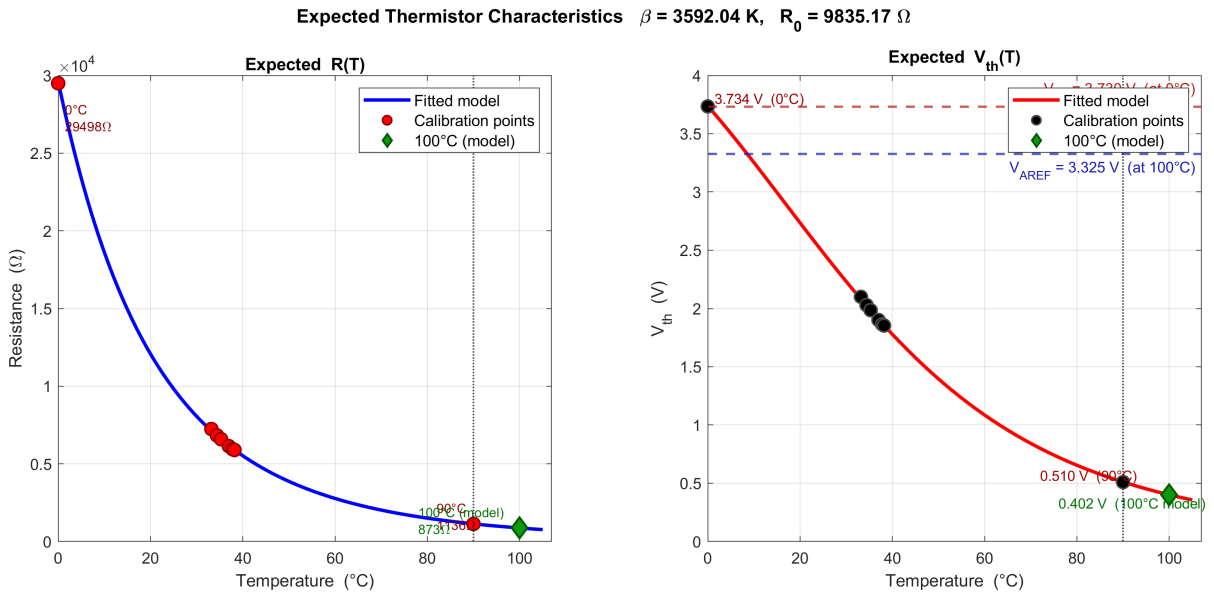


Figure 17: Expected thermistor characteristics from the fitted model over 0–100 °C. *Left:* resistance $R(T)$. *Right:* divider output $V_{\text{th}}(T)$ for $V_{cc} = 5\,\text{V}$ and $R_1 = 10\,\text{k}\Omega$. The calibration points are shown on both plots.

Figure 17 shows the expected $R(T)$ and $V_{\text{th}}(T)$ curves from the fitted model. The eight calibration points lie close to both curves, which supports the model inside the calibrated range.

3.9. Calculation of the Design Voltages

Both design voltages are calculated from the calibration data and the fitted model.

3.9.1. Step 1: Ice Bath $\Rightarrow V_{\text{ref}} = 3.73\,\text{V}$

The plateau in Figure 8 gives $V_{\text{th}}(0\,^\circ\text{C}) = 3.7300\,\text{V}$. Using the measured steady-state resistance in the divider equation (Equation (6)) gives almost the same value:

$$V_{\text{th}}(0\,^\circ\text{C}) = 5.0 \times \frac{29{,}498}{10{,}000 + 29{,}498} = 3.7341\,\text{V} \tag{17}$$

The two values agree within 4 mV. For the difference amplifier output to be 0 V in the ice bath, its non-inverting input has to be at this voltage:

$$V_{\text{out}}(0\,^\circ\text{C}) = V_{\text{ref}} - V_{\text{th}}(0\,^\circ\text{C}) = 3.73 - 3.73 = 0\,\text{V} \tag{18}$$

So potentiometer 1 was set to $V_{\text{ref}} = 3.73\,\text{V}$.

3.9.2. Step 2: Extrapolation to 100 °C $\Rightarrow V_{\text{AREF}} = 3.325\,\text{V}$

Using the measured 90 °C point to set the ADC reference would limit the ADC range to 0–90 °C. Instead, the fitted model was evaluated at 100 °C, so that ADC full scale corresponds to the top of the intended 0–100 °C range.

Step 2a: Resistance at 100 °C from the model.

Substituting $T = 373.15\,\mathrm{K}$ into Equation (2):

$$R(100\,^\circ\mathrm{C}) = 9835.46 \times \exp\left[3578.29 \times \left(\frac{1}{373.15} - \frac{1}{298.15}\right)\right] = 9835.46 \times e^{-2.412} = 881.4\,\Omega \tag{19}$$

Step 2b: Divider voltage at 100 °C.

$$V_{\mathrm{th}}(100\,^\circ\mathrm{C}) = 5.0 \times \frac{881.4}{10{,}000 + 881.4} = 0.4050\,\mathrm{V} \tag{20}$$

Step 2c: Difference amplifier output at 100 °C.

$$V_{\mathrm{out}}(100\,^\circ\mathrm{C}) = V_{\mathrm{ref}} - V_{\mathrm{th}}(100\,^\circ\mathrm{C}) = 3.730 - 0.405 = 3.325\,\mathrm{V} \tag{21}$$

This value was used as $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ on potentiometer 2, which gives the following ADC mapping:

- ADC $= 0 \longleftrightarrow V_{\mathrm{out}} = 0\,\mathrm{V} \longleftrightarrow 0\,^\circ\mathrm{C}$
- ADC $= 1023 \longleftrightarrow V_{\mathrm{out}} = 3.325\,\mathrm{V} \longleftrightarrow 100\,^\circ\mathrm{C}$ (nominal)

3.9.3. Voltage Table at All Calibration Points

Table 2 lists the divider voltage and the expected amplifier output at the eight calibration points, plus the 100 °C model value that defines AREF.

Table 2: Expected voltages at each calibration point. The last row (100 °C) comes from the fitted model and defines V_{AREF}.

Pt	T (°C)	R_{th} (Ω)	V_{th} (V)	$V_{\mathrm{out}} = V_{\mathrm{ref}} - V_{\mathrm{th}}$ (V)
1	0.0	29 498.05	3.7341	≈0.000
2	33.2	7 241.27	2.1000	1.630
3	34.4	6 832.37	2.0295	1.701
4	35.3	6 585.11	1.9852	1.745
5	37.0	6 135.16	1.9012	1.829
6	37.8	5 944.50	1.8641	1.866
7	38.2	5 892.11	1.8538	1.876
8	90.0	1 135.84	0.5100	3.220
100.0 (model)		*881.4*	*0.405*	*3.325* ← **AREF**

4. Circuit Design and Signal Conditioning

4.1. System Overview

The measurement chain has four stages in series. Stage 1 is the thermistor voltage divider, which turns resistance into voltage. Stage 2 is a potentiometer that gives a fixed 3.73 V

reference. Stage 3 is a unity-gain difference amplifier that subtracts the thermistor voltage from this reference, so the output is 0 V at 0 °C and 3.325 V at 100 °C. Stage 4 puts 3.325 V on the Arduino AREF pin, so the 0–100 °C range uses all 1023 ADC steps.

4.2. Stage 1: Thermistor Voltage Divider

4.2.1. Circuit Description

The 10 kΩ series resistor (R_1) goes between the Arduino 5 V rail and one lead of the thermistor, and the other thermistor lead goes to GND. The node between them carries the signal voltage V_{th} (Equation (6)).

4.2.2. Wiring on the Breadboard

- **5 V rail → R_1 (10 kΩ):** one leg of the resistor goes into a row connected to the 5 V rail.
- **R_1 → thermistor:** the other leg of R_1 and one thermistor lead share the same row. This row is the V_{th} node.
- **Thermistor → GND rail:** the second thermistor lead goes to the GND rail.
- **V_{th} node → op-amp inverting input:** a short jumper connects the V_{th} node to the 1 MΩ input resistor ($R_{\text{in,inv}}$) of the difference amplifier in Stage 3.

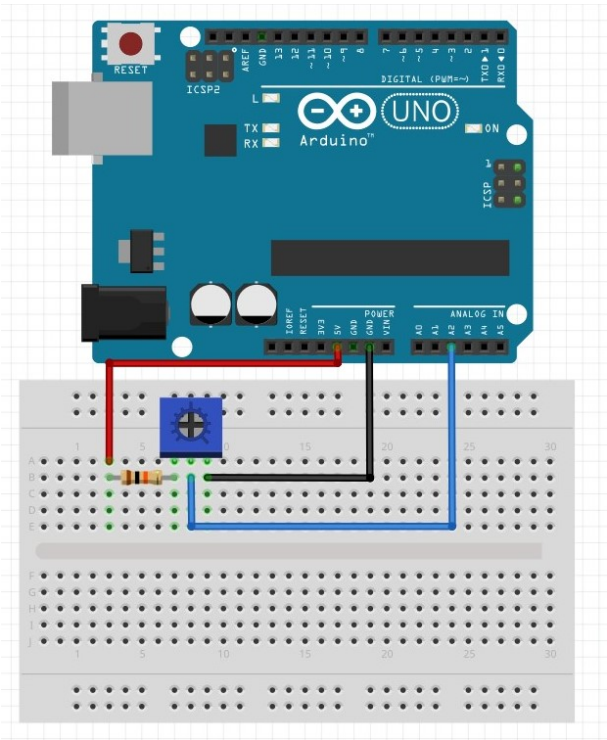


Figure 18: Stage 1 thermistor voltage divider wiring. The V_{th} node is connected by a jumper to the inverting input of Stage 3.

4.3. Stage 2: Reference Voltage Generator (3.73 V)

4.3.1. Circuit Description

A trimmer potentiometer provides the 3.73 V reference V_{ref} for the difference amplifier. With its two ends connected to 5 V and GND, it works as an adjustable divider (Equation (7)), and the wiper can be set anywhere between 0 and 5 V.

4.3.2. Wiring on the Breadboard

- **One end terminal → 5 V rail.**
- **Other end terminal → GND rail.**
- **Wiper → op-amp non-inverting input:** the wiper (centre pin) goes to the 1 MΩ input resistor ($R_{\text{in,non}}$) on the non-inverting side of Stage 3.
- **Wiper → Arduino A2 (optional monitor):** a second jumper from the wiper to A2 lets the live voltage reader (Appendix B) show the wiper voltage while it is being adjusted.

4.3.3. Setting the Potentiometer with the Live Voltage Reader

While building the circuit, a short Arduino sketch printed the wiper voltage to the Serial Monitor at 2 Hz, and the screw was turned slowly until the reading settled at 3.73 V. Only the voltage column matters here; the resistance column is only meaningful for the thermistor divider (see the note in Appendix B).

Listing 4: Core of the live voltage reader used to set both potentiometers.

```
int   adc   = analogRead(PIN_ANALOG);
float Vout  = (float)adc * VREF / (float)ADC_RES;
float R_pot = -1.0f;                          // -1 = out of range
if (Vout > 0.01f && Vout < VREF - 0.01f)
  R_pot = R_FIXED * (VREF / (VREF - Vout) - 1.0f);
Serial.print(Vout, 4); Serial.print(",");
Serial.println(R_pot, 2);
```

Serial Monitor output while setting the reference:

```
Vout[V], R_pot[Ohm]
4.4033,  58935.43
3.9345,  35002.55
3.7954,  28820.12
3.7354,  25701.34
3.7310,  25536.21
3.7300,  25491.80
3.7300,  25491.80   <- stable at target  3.730 V
```

Listing 5: Serial Monitor output while adjusting potentiometer 1. A new line is printed every 500 ms.

The same procedure was used to set potentiometer 2 to $V_{\text{AREF}} = 3.325$ V (Section 4.5).

4.4. Stage 3: Unity-Gain Difference Amplifier

4.4.1. Circuit Description and Pin Assignment

The op-amp is powered from a single 5 V supply: its positive supply pin goes to the 5 V rail and its negative supply pin to GND. Four 1 MΩ resistors form the difference amplifier of Figure 5:

- $R_{\mathbf{in,inv}}$ **(1 MΩ):** from the V_{th} node (Stage 1 output) to the inverting input (V−). It carries the thermistor signal.
- R_F **(1 MΩ):** from the output back to V−. Together with $R_{\text{in,inv}}$ it sets the inverting gain to $-R_F/R_{\text{in,inv}} = -1$.
- $R_{\mathbf{in,non}}$ **(1 MΩ):** from the potentiometer 1 wiper ($V_{\text{ref}} = 3.73\,\text{V}$) to the non-inverting input (V+).
- $R_{\mathbf{gnd}}$ **(1 MΩ):** from V+ to GND. This resistor is needed so that both inputs see the same resistor network. Without it, the two inputs would have different gains and the output would not be $V_{\text{ref}} - V_{\text{th}}$.

4.4.2. Wiring on the Breadboard

- The op-amp sits across the centre gap of the breadboard, so each row of pins is on its own side and no pins are shorted together.
- **Power:** op-amp $\text{V}_{\text{CC+}}$ pin → 5 V rail; $\text{V}_{\text{CC-}}$ pin → GND rail.
- **Inverting path:** V_{th} row $\xrightarrow{R_{\text{in,inv}}}$ V− pin row $\xrightarrow{R_F}$ output pin row.
- **Non-inverting path:** potentiometer 1 wiper row $\xrightarrow{R_{\text{in,non}}}$ V+ pin row $\xrightarrow{R_{\text{gnd}}}$ GND rail.
- **Output:** op-amp output row → Arduino A0 through a short jumper. This is the ADC input.

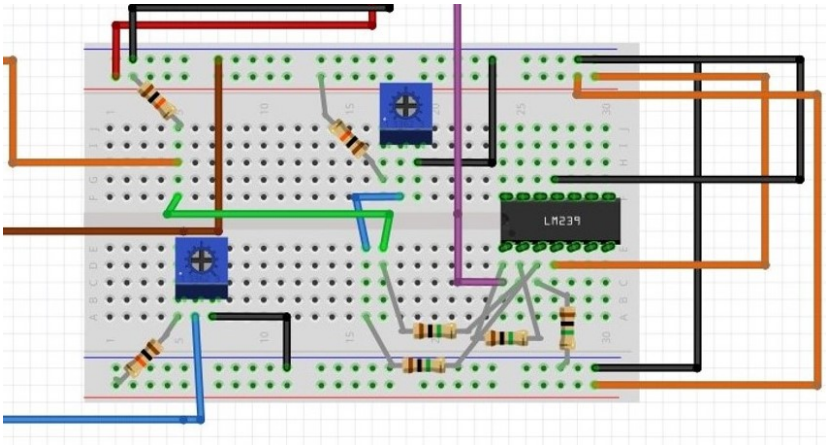


Figure 19: Stage 3 difference amplifier wiring. All four resistors are 1 MΩ. Inverting input: V_{th} from Stage 1. Non-inverting input: $V_{\text{ref}} = 3.73\,\text{V}$ from Stage 2. Output to Arduino A0.

4.4.3. Choice of 1 MΩ Resistors

The 1 MΩ value comes from the loading requirement (Section 2.3.4). The Thévenin impedance of the divider, $R_{\text{Th}} = R_1 \| R_{\text{th}}$, changes with temperature and is largest at 0 °C, where $R_{\text{th}} = 29{,}498\,\Omega$:

$$R_{\text{Th,max}} = \frac{10{,}000 \times 29{,}498}{10{,}000 + 29{,}498} = 7{,}468\,\Omega \tag{22}$$

Table 3: Thévenin source impedance of the thermistor divider at each calibration temperature.

T (°C)	R_{th} (Ω)	$R_{\text{Th}} = R_1 \| R_{\text{th}}$ (Ω)
0.0	29 498	7 468
33.2	7 241	4 200
34.4	6 832	4 059
35.3	6 585	3 971
37.0	6 135	3 802
37.8	5 945	3 728
38.2	5 892	3 708
90.0	1 136	1 020

4.4.4. Verification of the 10× Impedance Rule

Applying Equation (10) to the worst case:

$$10 \times R_{\text{Th,max}} = 10 \times 7{,}468 = 74{,}680\,\Omega \tag{23}$$

$$Z_{\text{in}} = 1{,}000{,}000\,\Omega \gg 74{,}680\,\Omega \quad \checkmark \quad (\text{margin factor} = 13.4) \tag{24}$$

4.4.5. Loading Error at 0 °C

Using Equation (9):

$$\varepsilon_V = 3.7341 \times \frac{7{,}468}{1{,}000{,}000} = 27.9\,\text{mV} \tag{25}$$

Converting to temperature with the divider sensitivity at 0 °C:

$$\left.\frac{dV_{\text{th}}}{dT}\right|_{0\,^\circ\text{C}} = V_{cc} \cdot \frac{R_1\,(dR/dT)}{(R_1 + R_{\text{th}})^2} = 5.0 \times \frac{10{,}000 \times (-1{,}415)}{(39{,}498)^2} = -45.3\,\text{mV/K} \tag{26}$$

$$\varepsilon_T = \frac{27.9\,\text{mV}}{45.3\,\text{mV/K}} \approx 0.62\,^\circ\text{C} \tag{27}$$

This is a fixed bias. With 100 kΩ resistors the error would be about 0.28 V (about 6 °C), and with 10 kΩ resistors it would be more than a volt.

4.4.6. Gain Stage vs. External ADC Reference

Another option would be to give the difference amplifier a gain of $G = 5.0/3.325 = 1.504$, so that its output covers the full 0–5 V ADC range and no AREF circuit is needed. This was considered and rejected for the following reasons:

1. **Output swing on a single supply.** The op-amp runs from a single 5 V rail, and even rail-to-rail op-amps stop some tens of millivolts below the supply, so an output of exactly 5.0 V cannot be reached.
2. **Non-standard resistor values.** $G = 1.504$ needs $R_F = 1.504\,R_{\text{in}}$, which is 1,504 kΩ for $R_{\text{in}} = 1\,\text{M}\Omega$. This is not an E24 or E96 value, so two resistors would have to be combined in series, which adds wiring and another tolerance error.
3. **Common-mode rejection.** The CMRR of a difference amplifier depends on how well the resistor ratios on its two inputs match. Matching a non-standard ratio built from combined resistors is harder than matching four equal resistors.
4. **The AREF approach is simpler.** Setting $V_{\text{AREF}} = 3.325\,\text{V}$ uses all 1023 ADC steps without changing the amplifier. The second potentiometer needs no extra IC, and the amplifier keeps its equal-resistor design.

4.5. Stage 4: External ADC Reference (3.325 V)

4.5.1. Circuit Description

A second trimmer potentiometer provides the 3.325 V external reference for the ADC. This value is the amplifier output that the model predicts at 100 °C (Equation (21)), so ADC full scale (count 1023) corresponds to 100 °C.

4.5.2. Wiring on the Breadboard

- **End terminals → 5 V and GND rails:** same as potentiometer 1.
- **Wiper → 5 kΩ protection resistor:** the wiper is connected to AREF through a 5 kΩ resistor. If the firmware runs without `analogReference(EXTERNAL)`, the external voltage is connected against the internal reference, and this resistor limits the current in that case.
- **5 kΩ resistor → AREF pin:** a jumper goes from the other end of the resistor to the AREF pin.
- **Wiper → Arduino A1 (optional monitor):** a second jumper from the wiper to A1 lets the live voltage reader show the voltage while it is being set.

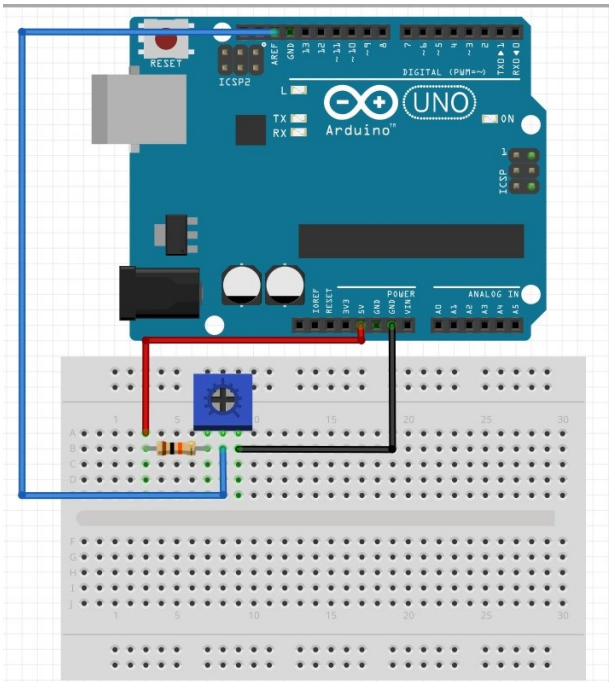


Figure 20: Stage 4 AREF wiring, with the 5 kΩ protection resistor between the potentiometer wiper and the AREF pin.

The resulting ADC step size is:

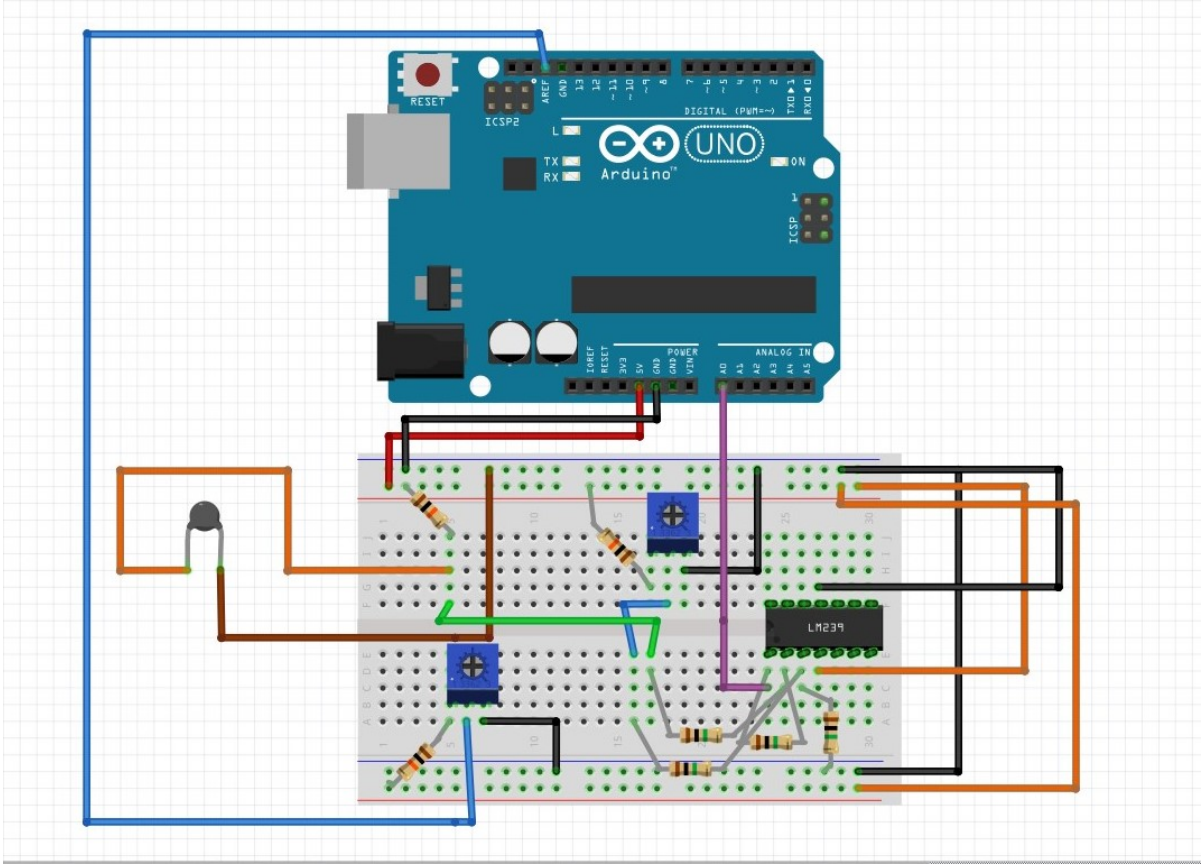
$$\Delta V = \frac{V_{\text{AREF}}}{1023} = \frac{3.325}{1023} = 3.25\,\text{mV/step} \tag{28}$$

compared with 4.89 mV per step with the default 5 V reference.

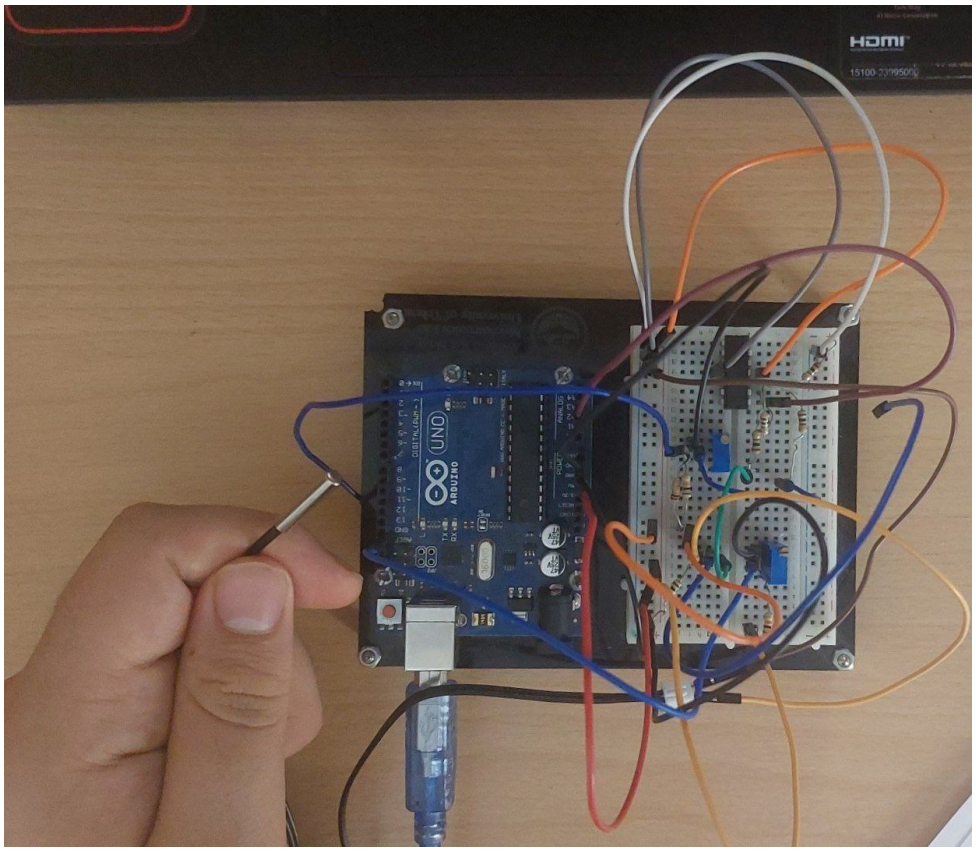
4.6. Full Assembled Circuit

Figure 21 shows the hand-drawn schematic and the assembled breadboard circuit. The schematic (Figure 21a) follows the signal from the 5 V supply through the voltage divider and the thermistor, then through the op-amp and its four 1 MΩ resistors, to the Arduino A0 input. The external AREF supply is drawn separately.

The breadboard (Figure 21b) follows the same order from left to right. The thermistor probe leaves the board at the far left toward the bath, the 10 kΩ resistor and potentiometer 1 (3.73 V reference) are on the left, the op-amp is in the middle with its four 1 MΩ resistors around it, potentiometer 2 and the 5 kΩ AREF resistor are on the right, and the Arduino Uno is connected at the far right.



(a) Full circuit schematic with all four stages, component values and net names.



(b) Assembled breadboard circuit.

Figure 21: Complete measurement circuit: (a) schematic and (b) breadboard. The breadboard follows the left-to-right order of the schematic. A walkthrough video is available at [4].

5. Firmware Implementation

5.1. AREF Safety Requirement

In the firmware, `analogReference(EXTERNAL)` has to be called at the start of `setup()`, before any `analogRead()`. If `analogRead()` runs while the default internal reference is selected and an external voltage is connected to AREF, the internal reference and the external source are connected together, which can damage the ADC. Calling `analogReference(EXTERNAL)` first avoids this. The firmware also discards the first reading after the reference is changed.

5.2. Signal-Chain Inversion Logic

The firmware undoes each stage of the signal chain in reverse order:

Listing 6: Signal-chain inversion functions. Each function undoes one hardware stage.

```
float reconstruct_Vtherm(float v_adc) {          // undo Stage 3
  return V_FIXED_REF - v_adc;
}

float compute_Rtherm(float v_therm) {            // undo Stage 1
  if (v_therm <= 0.01f) v_therm = 0.01f;
  if (v_therm >= 4.99f) v_therm = 4.99f;
  return R_SERIES * (v_therm / (VCC - v_therm));
}

float compute_Temperature(float R_therm) {       // beta model
  float inv_T = (1.0f / T0_K) + (logf(R_therm / R0) / BETA);
  return (1.0f / inv_T) - 273.15f;
}
```

The full chain is:

$$V_{\text{adc}} = \frac{\text{ADC}}{1023} \cdot V_{\text{AREF}} \xrightarrow{\text{undo Stage 3}} V_{\text{th}} = V_{\text{ref}} - V_{\text{adc}} \xrightarrow{\text{undo Stage 1}} R_{\text{th}} = R_1 \cdot \frac{V_{\text{th}}}{V_{cc} - V_{\text{th}}} \xrightarrow{\beta\text{-model}} T \tag{29}$$

5.3. Firmware Constants and Their Origins

Each constant in the firmware comes from a calibration step or a design value:

Table 4: Firmware constants and where they come from.

Constant	Value	Type	Source
`V_AREF`	3.325 V	Derived	Model extrap. at 100 °C (Eq. (21))
`V_FIXED_REF`	3.730 V	Measured	Ice bath SS mean (Fig. 8)
`R_SERIES`	10 000 Ω	Nominal	Fixed divider resistor
`VCC`	5.0 V	Nominal	Arduino 5 V rail
`R0`	9835.46 Ω	Fitted	MATLAB `polyfit` (Eq. (16))
`BETA`	3578.29 K	Fitted	MATLAB `polyfit` (Eq. (16))
`T0_K`	298.15 K	Fixed	Reference temperature (25 °C)

6. Results and Verification

6.1. Verification Procedure

After the final firmware was uploaded, the system was checked at two points:

1. **Ice bath (0 °C):** the probe was put back into the same ice bath used for calibration, and the Serial Monitor was watched until the reading settled. A correctly calibrated system should show $0 \pm \varepsilon_T$ °C, where $\varepsilon_T \approx 0.62$ °C is the expected loading error (Equation (27)).
2. **Mixed bath with thermometer (34.4 °C):** the probe was placed in a stable mixed bath together with the medical thermometer, and the two readings were compared.

6.2. Verification Results

Ice bath:

```
V_adc,   V_th,    R_th,        Temp_C
0.0161,  3.7139,  28836.59,    0.44
0.0161,  3.7139,  28836.59,    0.44
0.0163,  3.7137,  28824.11,    0.44
0.0161,  3.7139,  28836.59,    0.44
```

Listing 7: Serial Monitor output in the ice bath during the final check. The system reads +0.44 °C instead of 0.00 °C, which is within the predicted loading offset of 0.62 °C.

The +0.44 °C offset is within the predicted 0.62 °C loading error.

Mixed bath at 34.4 °C:

```
V_adc,   V_th,    R_th,        Temp_C
1.6884,  2.0416,  6922.11,     34.32
1.6884,  2.0416,  6922.11,     34.32
1.6933,  2.0367,  6887.82,     34.38
1.6884,  2.0416,  6922.11,     34.32
```

Listing 8: Serial Monitor output in the 34.4 °C mixed bath. The firmware reads 34.32 °C, while the medical thermometer reads 34.4 °C, a difference of 0.08 °C.

6.3. Summary of Verification Accuracy

Table 5: Final verification results.

Test	**Ref.** (°C)	**Firmware** (°C)	**Error** (°C)	**Reference type**
Ice bath	0.00	+0.44	+0.44	Thermodynamic fixed point
Mixed bath	34.4	34.32	−0.08	Medical thermometer

7. Discussion

7.1. Model Validity and Calibration Density

The β-parameter model is an approximation that works well over moderate temperature ranges. It captures the main exponential behaviour of NTC thermistors but leaves out higher-order terms that matter over ranges much wider than about 100 °C. For the 0–100 °C range used here, the two-parameter model is adequate [1].

A bigger issue is how unevenly the calibration points are spread. Six of the eight points lie within 5 °C (33–38 °C), and their $x_i = 1/T_i - 1/T_0$ values differ by only about $5 \times 10^{-5}\,\mathrm{K}^{-1}$, so they add little independent information. β and R_0 are therefore set mostly by the two end points at 0 °C and 90 °C. This does not hurt the accuracy at those two points, but the model between them is checked by only a few measurements. Additional points at 10, 20, 50 and 70 °C, measured with a thermocouple, would test the model over the whole range and make the fit residuals more meaningful.

7.2. ADC Resolution Improvement

Table 6: ADC use with the default 5 V reference and with the $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ external reference.

Parameter	**Default (5 V)**	**External (3.325 V)**
AREF voltage	5.000 V	3.325 V
Signal span (0–100 °C)	0–3.325 V	0–3.325 V
ADC steps used	680 / 1023 (66.5%)	1023 / 1023 (100%)
Voltage per step	4.89 mV	3.25 mV
Resolution improvement	–	+33.5%
Additional steps	–	+343 steps

7.3. Error Sources and Analysis

1. **Loading error (0.62 °C at 0 °C).** The 27.9 mV offset caused by the finite 1 MΩ input impedance (Equation (27)) is a fixed bias, so averaging cannot remove it. An instrumentation amplifier (Section 2.6) would remove it.
2. **Few and unevenly spread calibration points.** As discussed in Section 7.1, six of the eight points are within a 5 °C band, which makes the model less reliable outside that band. More points spread over 0–100 °C would reduce this error.
3. **Potentiometer setting accuracy.** Both V_{ref} and V_{AREF} were set by hand using the live voltage reader, which relies on the Arduino's own 10-bit ADC (about 4.89 mV per step at 5 V). A 10 mV error in V_{ref} shifts every reconstructed V_{th} by 10 mV, which is about 0.2 °C at 25 °C. Setting the voltages with a precision multimeter would reduce this error.
4. **Medical thermometer accuracy.** Clinical thermometers are designed for about 35–42 °C and may have a systematic error of ±0.3–0.5 °C near the edges of their range. The ice bath does not have this problem, since it is a fixed point.
5. **ADC quantisation.** At 3.25 mV per step and $|dV_{\text{th}}/dT| \approx 45.3\,\text{mV/K}$ at 0 °C (Equation (26)), the smallest temperature step the ADC can resolve is $3.25/45.3 \approx 0.07$ °C near 0 °C.
6. **Thermistor self-heating.** The 5 V supply drives about 250 μA through the thermistor at 25 °C, which dissipates about 0.6 mW and warms it slightly in still air. In a stirred water bath the heat transfer is high enough that this error is negligible.
7. **Op-amp output near 0 V.** On a single 5 V supply the op-amp output cannot reach exactly 0 V. Near 0 °C the output is only about 16 mV (Listing 7), so this may add a small offset there.
8. **Model extrapolation.** V_{AREF} was found by evaluating the model at 100 °C, 10 °C above the highest calibration point. Over such a short extrapolation the β-model behaves well, but any error in β has a slightly larger effect than inside the calibrated range. Measuring a point at 100 °C with a thermocouple would remove this uncertainty.
9. **Uncertainty of the 90 °C point.** The boiling-water temperature was estimated rather than measured, because it is outside the medical thermometer's range. Since β is set mostly by the 0 °C and 90 °C points (Section 7.1), an error in this value shifts β directly. Measuring this point with a thermocouple would remove this uncertainty.

8. Conclusion

A complete NTC thermistor thermometer was designed, calibrated and tested. The only reference instrument was a medical thermometer with a 32–43 °C range, so the intermediate calibration points are all close to body temperature. The 0 °C and 90 °C end points were taken without the thermometer, using an ice bath and boiling water. The 90 °C value was estimated from Tehran's altitude (about 1200–1500 m) and the cooling of the water after the kettle was unplugged.

The ice bath reading ($V_{\text{th}} = 3.73\,\text{V}$, Figure 8) set the difference amplifier reference V_{ref}. The ADC reference was not taken from the 90 °C measurement. Instead, the fitted β-model was evaluated at 100 °C to find the largest expected amplifier output (Equation (21)):

$$V_{\text{AREF}} = V_{\text{ref}} - V_{\text{th}}(100\,^{\circ}\text{C}) = 3.730 - 0.405 = 3.325\,\text{V}$$

This maps ADC full scale (count 1023) to 100 °C instead of the measured 90 °C, with a resolution of 3.25 mV per step, 33.5% finer than with the 5 V default. A gain stage that stretches the output to 0–5 V was not used: the single-supply op-amp cannot reach 5 V, the non-standard resistor values would make resistor matching harder, and the AREF approach uses the ADC just as fully with a simpler amplifier (Section 4.4.6).

The 1 MΩ amplifier resistors meet the 10× impedance rule with a margin of 13.4 (Equation (24)), which leaves a worst-case loading error of 0.62 °C at 0 °C. In the final test (Section 6.2), the system read +0.44 °C in the ice bath (Listing 7) and 34.32 °C in a bath where the medical thermometer read 34.4 °C (Listing 8), a difference of 0.08 °C.

Acknowledgements

I would like to thank Dr. Ali Sadighi for designing the course, providing the theory needed for the lab, and his clear teaching throughout the semester. I also thank the teaching assistants, Mohammad Jadidi and Erfun Bayat Sarmadi, for their help during the lab sessions.

References

[1] M. Robati, A. Mashhadireza, S. Sehhat, M. Jadidi, E. B. Sarmadi, "Lab Session 06: Temperature Measurement System," *Mechatronics Laboratory Manual*, University of Tehran, ver. 1.2, Dec. 2025.

[2] D. G. Alciatore, *Introduction to Mechatronics and Measurement Systems*, 4th ed., McGraw-Hill, 2012.

[3] Analog Devices, "AD620 Low Cost Instrumentation Amplifier," Data Sheet. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/AD620.pdf

[4] R. Amini, "NTC Thermistor Temperature Measurement System: Circuit Walkthrough and Experimental Demonstration," *Supplementary Video, Lab Session 06*, University of Tehran, Feb. 2026. [Online]. Available: Google Drive (click to open video)

[5] R. Amini, "NTC Thermistor Temperature Measurement and Calibration System," GitHub repository, 2026. [Online]. Available: https://github.com/RoozbehAmini/NTC-Thermistor-Temperature-Measurement-and-Calibration-System

Appendices

All the code below is also available in the project's GitHub repository [5].

A. Arduino Calibration Logger (50 Hz, 60 s)

Listing 9: Calibration logger (`Arduino/calibration_logger`). Streams `t_s, adc, volt_V, R_therm_Ohm` at 50 Hz for 60 s, then stops.

```
/*
 * Calibration Logger (50 Hz, 60 s)
 * Measurement Systems Lab, Session 06 - University of Tehran
 *
 * Wiring (calibration stage, no amplifier yet):
 *   5V --[10k]--+--[NTC thermistor]-- GND
 *               |
 *               +--> A0
 *
 * Samples A0 every 20 ms for 60 s and streams CSV over serial:
 *   t_s, adc, volt_V, R_therm_Ohm
 * then stops. Opening the serial port (e.g. from the MATLAB script)
 * resets the Uno, which starts a fresh 60 s log.
 */

const int           THERM_PIN   = A0;
const float         VREF        = 5.0f;      // default ADC reference (5 V rail)
const float         R_FIXED     = 10000.0f;  // series resistor on the 5 V side
const unsigned long DURATION_MS = 60000UL;   // 60 s record
const unsigned long Ts_ms       = 20UL;      // 20 ms -> 50 Hz

void setup() {
  Serial.begin(115200);
  Serial.println("t_s,adc,volt_V,R_therm_Ohm");
}

void loop() {
  static bool done = false;
  if (done) return;

  unsigned long t0   = millis();
  unsigned long next = t0;

  while (millis() - t0 < DURATION_MS) {
    if (millis() >= next) {
      next += Ts_ms;
      float t_s = (millis() - t0) / 1000.0f;
      int   adc = analogRead(THERM_PIN);
      float v   = adc * VREF / 1023.0f;

      // Divider inversion (thermistor on the GND side): R = R1 * V / (Vcc - V)
      float Rth = (v > 0.01f && v < VREF - 0.01f)
                    ? R_FIXED * v / (VREF - v)
```

```
                    : 0.0f;

      Serial.print(t_s, 4); Serial.print(",");
      Serial.print(adc);    Serial.print(",");
      Serial.print(v, 6);   Serial.print(",");
      Serial.println(Rth, 2);
    }
  }
  done = true;
}
```

B. Live Voltage Reader

Listing 10: Live voltage reader (`Arduino/live_voltage_reader`), used to set both potentiometers. Prints `Vout[V], R_pot[Ohm]` every 500 ms.

```
/*
 * Live Voltage Reader
 * Measurement Systems Lab, Session 06 - University of Tehran
 *
 * Used to set the two trimmer potentiometers:
 *   Pot 1 (diff-amp reference, target 3.730 V): wiper -> A2
 *   Pot 2 (ADC reference,      target 3.325 V): wiper -> A1
 * Change PIN_ANALOG to match the pot you are adjusting.
 *
 * Prints  Vout[V], R_pot[Ohm]  every 500 ms (2 Hz).
 *
 * Note: R_pot uses the divider formula R = R_FIXED * V / (Vcc - V), so it
 * is only meaningful when an unknown resistor sits under a 10k fixed
 * resistor (e.g. the thermistor divider). When setting a pot, read Vout only.
 */

const int   PIN_ANALOG = A2;       // A2 = pot 1, A1 = pot 2
const float VREF       = 5.0f;
const int   ADC_RES    = 1023;
const float R_FIXED    = 10000.0f;

void setup() {
  analogReference(DEFAULT);
  Serial.begin(9600);
  while (!Serial) { ; }
  Serial.println("Vout[V],R_pot[Ohm]");
}

void loop() {
  int   adc   = analogRead(PIN_ANALOG);
  float Vout  = (float)adc * VREF / (float)ADC_RES;
  float R_pot = -1.0f;                                  // -1 = out of range
  if (Vout > 0.01f && Vout < VREF - 0.01f)              // avoid divide-by-zero at full scale
    R_pot = R_FIXED * (VREF / (VREF - Vout) - 1.0f);

  Serial.print(Vout, 4); Serial.print(",");
```

```
  Serial.println(R_pot, 2);
  delay(500);
}
```

C. MATLAB Calibration Acquisition Script

Listing 11: `Matlab/acquire_calibration_point.m`: records one 60 s calibration point, extracts the steady-state mean, saves the raw trace, appends the point to the master list and plots the traces.

```
% ACQUIRE_CALIBRATION_POINT
% Measurement Systems Lab, Session 06 - University of Tehran
%
% Records one 60 s calibration point from the Arduino calibration logger
% (arduino/calibration_logger), extracts the steady-state mean from the
% last 10 s, saves the raw trace, appends the point to the master list,
% and plots the voltage and resistance traces.
%
% Usage: upload calibration_logger to the Uno, put the probe in the bath,
% wait until the reference thermometer is stable, then run this script.
% Opening the port resets the Uno, which starts a fresh 60 s log.
%
% Requires MATLAB R2019b+ (serialport).

clear; clc; close all;

port       = "COM4";      % change to your Arduino port
baud       = 115200;
duration   = 60;          % s
steady_win = 10;          % s, steady-state window at the end of the record

% Output folder: ../data relative to this script (falls back to pwd)
script_dir = fileparts(mfilename('fullpath'));
if isempty(script_dir), script_dir = pwd; end
data_dir = fullfile(script_dir, '..', 'data');
if ~exist(data_dir, 'dir'), mkdir(data_dir); end

ref_temp = input('>> Enter bath temperature (deg C): ');

s = serialport(port, baud);
configureTerminator(s, "LF");
flush(s); pause(1);
if s.NumBytesAvailable > 0, readline(s); end   % drop header / partial line

count = 0;
buf   = zeros(duration*60, 3);   % headroom above 50 Hz x 60 s
tic;
while toc < duration
    if s.NumBytesAvailable > 0
        vals = sscanf(readline(s), "%f,%d,%f,%f");   % t_s, adc, volt_V, R_therm_Ohm
        if numel(vals) == 4
            count = count + 1;
```

```
            buf(count, :) = [vals(1), vals(3), vals(4)];
        end
    end
end
delete(s); clear s;

T = buf(1:count, 1);
V = buf(1:count, 2);
R = buf(1:count, 3);

%% Steady-state extraction
t_max  = max(T);
ss     = T >= (t_max - steady_win);
v_mean = mean(V(ss));
r_mean = mean(R(ss));
r_std  = std(R(ss));

%% Save full time series
tag      = strrep(sprintf('%.1f', ref_temp), '.', 'p');
raw_file = fullfile(data_dir, sprintf('cal_raw_%sC.csv', tag));
fid = fopen(raw_file, 'w');
fprintf(fid, 't_s,volt_V,R_therm_Ohm\n');
for k = 1:count
    fprintf(fid, '%.4f,%.6f,%.2f\n', T(k), V(k), R(k));
end
fclose(fid);

%% Append to master list
master = fullfile(data_dir, 'calibration_master_list.csv');
if ~isfile(master)
    fid = fopen(master, 'w');
    fprintf(fid, 'T_C,V_mean,R_mean\n');
    fclose(fid);
end
fid = fopen(master, 'a');
fprintf(fid, '%.2f,%.6f,%.6f\n', ref_temp, v_mean, r_mean);
fclose(fid);

%% Annotated plots
figure('Color', 'w', 'Position', [100 100 1200 500]);
subplot(2,1,1);
plot(T, V, 'Color', [.75 .75 .75]); hold on;
plot(T, movmean(V, 50), 'b', 'LineWidth', 1.8);   % 50 samples = 1 s
yl = ylim;
patch([t_max-steady_win t_max t_max t_max-steady_win], ...
      [yl(1) yl(1) yl(2) yl(2)], [0.18 0.78 0.18], ...
      'FaceAlpha', .12, 'EdgeColor', 'none');
yline(v_mean, 'r--', 'LineWidth', 1.6, ...
      'Label', sprintf('V_{ss}=%.4f V', v_mean), ...
      'LabelHorizontalAlignment', 'left', 'LabelVerticalAlignment', '
         bottom');
ylabel('V_{th} (V)'); grid on; box on;
legend({'Raw 50 Hz', 'Smoothed 1 s', 'Steady window'}, 'Location', 'best
   ');
title(sprintf('Voltage trace at %.1f degC', ref_temp));

subplot(2,1,2);
plot(T, R, 'Color', [.85 .33 .10], 'LineWidth', 1.4); hold on;
```

```
yl2 = ylim;
patch([t_max-steady_win t_max t_max t_max-steady_win], ...
      [yl2(1) yl2(1) yl2(2) yl2(2)], [0.18 0.78 0.18], ...
      'FaceAlpha', .12, 'EdgeColor', 'none');
yline(r_mean, 'k--', 'LineWidth', 1.6, ...
      'Label', sprintf('R_{ss}=%.2f Ohm', r_mean), ...
      'LabelHorizontalAlignment', 'left', 'LabelVerticalAlignment', '
         bottom');
xlabel('Time (s)'); ylabel('R_{th} (Ohm)'); grid on; box on;
title(sprintf('Resistance trace at %.1f degC', ref_temp));
sgtitle(sprintf('SS: V=%.4fV | R=%.2fOhm | sigma=%.3fOhm', ...
        v_mean, r_mean, r_std), 'FontWeight', 'bold');

fprintf('Done: %.1fC | %.4fV | %.2fOhm\n', ref_temp, v_mean, r_mean);
```

D. MATLAB Model Fitting and Characteristic Plots

Listing 12: `Matlab/fit_beta_model.m`: loads the calibration master list, fits the linearised β model, prints the fit residuals, extrapolates to 100 °C to find V_{AREF}, and saves the calibration and characteristic plots.

```
% FIT_BETA_MODEL
% Measurement Systems Lab, Session 06 - University of Tehran
%
% Loads the calibration master list, fits the linearised beta model
%   ln(R) = ln(R0) + beta * (1/T - 1/T0)
% by least squares, extrapolates to 100 degC to get V_AREF, and saves
    the
% calibration-curve and expected-characteristics figures.
%
% Data: ../data/calibration_master_list.csv (written by
% acquire_calibration_point.m). If that file is missing, the eight
% calibration points from the lab report are used.
%
% Requires MATLAB R2023a+ (xregion); exportgraphics needs R2020a+.

clear; clc; close all;

%% Setup
script_dir = fileparts(mfilename('fullpath'));
if isempty(script_dir), script_dir = pwd; end
save_dir = fullfile(script_dir, '..', 'figures');
if ~exist(save_dir, 'dir'), mkdir(save_dir); end
DPI = 300;

%% Load data
csv_path = fullfile(script_dir, '..', 'data', 'calibration_master_list.
    csv');
if isfile(csv_path)
    data   = readmatrix(csv_path);
    T_C    = data(:,1);
    R_meas = data(:,3);
    fprintf('Loaded %d points from %s\n', numel(T_C), csv_path);
else
    T_C    = [0.0; 33.2; 34.4; 35.3; 37.0; 37.8; 38.2; 90.0];
    R_meas = [29498.05; 7241.27; 6832.37; 6585.11; ...
```

```
                6135.16;  5944.50; 5892.11; 1135.84];
    fprintf('Master list not found, using report Table 1 data\n');
end
[T_C, ix] = sort(T_C);
R_meas    = R_meas(ix);
T_K       = T_C + 273.15;
T0        = 298.15;
Vcc = 5.0;  R1 = 10000;  Vref = 3.730;

%% Beta-model fit
X    = (1 ./ T_K) - (1 / T0);
Y    = log(R_meas);
p    = polyfit(X, Y, 1);
Beta = p(1);
R0   = exp(p(2));
fprintf('Beta = %.4f K\nR0   = %.4f Ohm\n', Beta, R0);

% Residuals in degC (how well the model reproduces each point)
T_fit_C = 1 ./ (1/T0 + log(R_meas/R0)/Beta) - 273.15;
fprintf('\n  T_ref (C)   T_model (C)   error (C)\n');
fprintf('  %8.2f   %10.3f   %+9.3f\n', [T_C, T_fit_C, T_fit_C - T_C]');

%% Extrapolate to 100 degC
T_extrap = 100;
R_100    = R0 * exp(Beta * (1/(T_extrap + 273.15) - 1/T0));
Vth_100  = Vcc * R_100 / (R1 + R_100);
Vout_100 = Vref - Vth_100;
fprintf('\nAt 100 degC (model extrapolation):\n');
fprintf('  R(100)    = %.2f Ohm\n', R_100);
fprintf('  Vth(100)  = %.4f V\n', Vth_100);
fprintf('  Vout(100) = %.4f V --> VAREF = %.3f V\n', Vout_100, Vout_100)
    ;

%% Model curve
Tc         = linspace(0, 105, 600)';
Tk         = Tc + 273.15;
Xp         = (1 ./ Tk) - (1 / T0);
Rm         = R0 .* exp(Beta .* Xp);
Vth_model  = Vcc .* Rm ./ (R1 + Rm);
Vout_model = Vref - Vth_model;
Vth_data   = Vcc .* R_meas ./ (R1 + R_meas);
Vout_data  = Vref - Vth_data;

idx0  = find(T_C == 0, 1);
idx90 = find(T_C == 90, 1);

%% --- FIGURE 1: Calibration Curve ---
hf1 = figure('Color', 'w', 'Position', [50 50 1000 560], 'Visible', 'on'
    );
ax1 = axes(hf1); hold on;

h_fit  = plot(Tc, Rm, 'b-', 'LineWidth', 2.2);
h_data = scatter(T_C, R_meas, 80, 'r', 'filled', ...
                 'MarkerEdgeColor', [0.55 0 0], 'LineWidth', 1.2);
h_ext  = scatter(T_extrap, R_100, 100, [0 0.6 0], 'd', 'filled', ...
                 'MarkerEdgeColor', [0 0.35 0], 'LineWidth', 1.5);
text(T_extrap + 1.5, R_100 + 300, ...
     sprintf('100 degC (model)\nR = %.0f Ohm', R_100), ...
```

```
    'FontSize', 9, 'Color', [0 0.45 0]);

if ~isempty(idx0)
    text(1, R_meas(idx0) - 2200, sprintf('0 degC\n%.0f Ohm', R_meas(idx0
        )), ...
        'FontSize', 9, 'Color', [0.5 0 0]);
end
if ~isempty(idx90)
    text(83, R_meas(idx90) + 500, sprintf('90 degC %.0f Ohm', R_meas(
        idx90)), ...
        'FontSize', 9, 'Color', [0.5 0 0]);
end

xregion(33, 38.5, 'FaceColor', [1 1 0], 'FaceAlpha', 0.15, ...
        'HandleVisibility', 'off');
text(35.5, 18000, '33-38 degC cluster', ...
    'HorizontalAlignment', 'center', 'FontSize', 9, 'Color', [0.5 0.4
        0]);

xlabel('Temperature (degC)', 'FontSize', 12);
ylabel('Thermistor Resistance (Ohm)', 'FontSize', 12);
title(sprintf('Calibration Curve    Beta = %.2f K,    R0 = %.2f Ohm',
    ...
     Beta, R0), 'FontSize', 13, 'FontWeight', 'bold');
legend([h_fit, h_data, h_ext], ...
       'Fitted beta-model', ...
       'Calibration points', ...
       '100 degC extrapolation (AREF basis)', ...
       'Location', 'northeast', 'FontSize', 10);
grid on; box on;
set(gca, 'FontSize', 10);

exportgraphics(hf1, fullfile(save_dir, 'fig_fitted_curve.png'), ...
               'Resolution', DPI);
fprintf('Saved: fig_fitted_curve.png\n');
close(hf1);

%% --- FIGURE 2: Expected R(T) and V_th(T) ---
hf2 = figure('Color', 'w', 'Position', [50 50 1200 500], 'Visible', 'on'
    );

subplot(1,2,1); hold on;
plot(Tc, Rm, 'b-', 'LineWidth', 2.2);
scatter(T_C, R_meas, 65, 'r', 'filled', 'MarkerEdgeColor', 'k');
xlabel('Temperature (degC)', 'FontSize', 12);
ylabel('Resistance (Ohm)', 'FontSize', 12);
title('Expected R(T)', 'FontSize', 12, 'FontWeight', 'bold');
legend({'Fitted model', 'Calibration data'}, 'Location', 'northeast',
    ...
       'FontSize', 10);
grid on; box on; set(gca, 'FontSize', 10);

subplot(1,2,2); hold on;
plot(Tc, Vth_model, 'r-', 'LineWidth', 2.2);
scatter(T_C, Vth_data, 65, 'k', 'filled');
if ~isempty(idx0)
    text(1.5, Vth_data(idx0) + 0.06, ...
         sprintf('%.2f V (0 degC)', Vth_data(idx0)), ...
```

```
        'FontSize', 9, 'Color', [0 0.4 0]);
end
if ~isempty(idx90)
    text(72, Vth_data(idx90) + 0.06, ...
        sprintf('%.2f V (90 degC)', Vth_data(idx90)), ...
        'FontSize', 9, 'Color', [0 0.4 0]);
end
scatter(T_extrap, Vth_100, 80, [0 0.6 0], 'd', 'filled', ...
    'MarkerEdgeColor', [0 0.35 0]);
text(T_extrap - 18, Vth_100 - 0.06, ...
    sprintf('%.3f V (100 degC model)', Vth_100), ...
    'FontSize', 9, 'Color', [0 0.4 0]);
xlabel('Temperature (degC)', 'FontSize', 12);
ylabel('V_{th} (V)', 'FontSize', 12);
title('Expected V_{th}(T)', 'FontSize', 12, 'FontWeight', 'bold');
legend({'Fitted model', 'Calibration data'}, 'Location', 'northeast', ...
    'FontSize', 10);
grid on; box on; set(gca, 'FontSize', 10);

sgtitle(sprintf('Expected Thermistor Characteristics    Beta = %.2f K,  R0 = %.2f Ohm', ...
    Beta, R0), 'FontSize', 12, 'FontWeight', 'bold');

exportgraphics(hf2, fullfile(save_dir, 'fig_expected_curves.png'), ...
    'Resolution', DPI);
fprintf('Saved: fig_expected_curves.png\n');
close(hf2);
```

E. Final Temperature Measurement Firmware

Listing 13: Final firmware (`Arduino/temperature_firmware`). `analogReference(EXTERNAL)` is called first. Reports V_{adc}, V_{th}, R_{th} and T at 2 Hz. The calibration constants come from the MATLAB fit (Appendix D), and $V_{\mathrm{AREF}} = 3.325\,\mathrm{V}$ comes from the model extrapolation to 100 °C (Eq. (21)).

```
/*
 * Final Temperature Measurement Firmware
 * Measurement Systems Lab, Session 06 - University of Tehran
 *
 * Forward signal chain:
 *   5V --[10k]--+--[Thermistor]-- GND
 *               |
 *              Vth --> Diff-Amp (V-)
 *   Pot-1 (3.730 V) --> Diff-Amp (V+)
 *   Diff-Amp output --> A0
 *   Pot-2 (3.325 V) --> [5k] --> AREF
 *
 * Firmware inversion:
 *   V_adc = (ADC / 1023) * V_AREF          (undo ADC scaling)
 *   V_th  = V_FIXED_REF - V_adc            (undo Stage 3 diff amp)
 *   R_th  = R_SERIES * V_th / (Vcc - V_th) (undo Stage 1 divider)
 *   T[C]  = 1 / (1/T0 + ln(R/R0)/beta) - 273.15
 *
 * V_AREF = 3.325 V derived from the beta model at 100 C:
 *   R(100 C) = 881.4 Ohm -> Vth(100 C) = 0.405 V
```

```
 *    Vout(100 C) = 3.730 - 0.405 = 3.325 V -> AREF
 *
 * AREF note: the AREF pin has an internal ~32k resistance, so the 5k
 * protection resistor forms a divider with it. Set V_AREF to the
    voltage
 * measured with a multimeter AT THE AREF PIN while this sketch is
    running,
 * not the voltage at the pot wiper.
 */

// --- Calibration constants (from MATLAB beta-model fit) ---
const float V_AREF      = 3.325f;    // model extrapolation to 100 C (
   see AREF note)
const float V_FIXED_REF = 3.730f;    // Pot-1: V_th at 0 C (ice bath)
const float R_SERIES    = 10000.0f;
const float VCC         = 5.0f;
const float R0          = 9835.46f;  // Ohm at 25 C
const float BETA        = 3578.29f;  // K
const float T0_K        = 298.15f;   // 25 C

const int SENSOR_PIN = A0;

float reconstruct_Vtherm(float v_adc);
float compute_Rtherm(float v_therm);
float compute_Temperature(float R_therm);

void setup() {
  analogReference(EXTERNAL);         // MUST be first, before any
     analogRead()
  analogRead(SENSOR_PIN);            // discard first reading after
     reference change
  Serial.begin(9600);
  while (!Serial) { ; }
  Serial.println("V_adc,V_th,R_th,Temp_C");
}

void loop() {
  int   adc    = analogRead(SENSOR_PIN);
  float v_adc  = (float)adc / 1023.0f * V_AREF;
  float v_therm = reconstruct_Vtherm(v_adc);
  float R_therm = compute_Rtherm(v_therm);
  float tempC  = compute_Temperature(R_therm);

  Serial.print(v_adc, 4);   Serial.print(",");
  Serial.print(v_therm, 4); Serial.print(",");
  Serial.print(R_therm, 2); Serial.print(",");
  Serial.println(tempC, 2);

  delay(500);                        // 2 Hz
}

float reconstruct_Vtherm(float v_adc) {
  return V_FIXED_REF - v_adc;
}

float compute_Rtherm(float v_therm) {
  if (v_therm <= 0.01f) v_therm = 0.01f;
  if (v_therm >= 4.99f) v_therm = 4.99f;
```

```
74   return R_SERIES * (v_therm / (VCC - v_therm));
75 }
76
77 float compute_Temperature(float R_therm) {
78   float inv_T = (1.0f / T0_K) + (logf(R_therm / R0) / BETA);
79   return (1.0f / inv_T) - 273.15f;
80 }
```